



UNIVERSIDAD ANDRÉS BELLO

FACULTAD DE INGENIERÍA

INGENIERÍA CIVIL INFORMÁTICA

ASISTENTE DE VOZ DE ESCRITORIO LOCAL Y PRIVADO PARA WINDOWS EN HARDWARE MODESTO (≤ 4 GB DE VRAM) MEDIANTE UN MODELO GEMMA 4 E2B FINE-TUNEADO

Proyecto de Título para optar al título de Ingeniero Civil en Informática

Autor: Emmanuel Villacura Arancibia

Profesor guía: Nicolás Caselli

Asignatura: INSW410 — Portafolio de Proyectos

Viña del Mar, Chile

2026

Tabla de contenido

Tabla de contenido	2
Declaración de Originalidad y Uso de Inteligencia Artificial	5
Resumen.....	5
Abstract.....	6
1. Introducción	7
2. Fundamentación del Problema.....	10
2.1. Contexto.....	10
2.2. Situación Actual	11
2.3. Técnica de Identificación de la Problemática: Diagrama de Ishikawa	13
2.4. Descripción de Causas	15
Categoría Tecnología / Máquina (causa principal)	15
Categoría Costo (causa principal, factor externo)	16
Categoría Privacidad (causa principal).....	16
Categoría Conectividad (causa secundaria, factor externo).....	17
Categoría Idioma / Usuario (causa secundaria)	17
Categoría Método (causa principal, factor interno).....	17
3. Alcance del Proyecto	19
3.1. Objetivo General	19
3.2. Objetivos Específicos.....	19
3.3. Métricas de los Objetivos (Tabla de Justificación)	20
3.3.1. Modelo de Negocio (CANVAS).....	22
3.4. Alternativas de Solución.....	24
3.4.1. Arquitectura general	24
3.4.2. Selección de modelo y cuantización (datos de VRAM medidos).....	25



3.4.3. Otras decisiones técnicas respaldadas por medición	25
3.5. Alcances y Limitaciones	26
3.5.1. Lo que el sistema SÍ hace (alcances)	26
3.5.2. Lo que el sistema NO hace o no garantiza (limitaciones)	26
3.6. Factibilidad Técnica	27
3.7. Factibilidad Económica	28
3.7.1. Costo operativo comparado	28
3.7.2. Viabilidad de licencias (vendibilidad)	28
3.8. Factibilidad Social	29
3.8.1. Privacidad	29
3.8.2. Accesibilidad e inclusión	29
3.8.3. Democratización y universalidad	30
4. Propuesta de Solución	31
4.1. Descripción de la solución	31
4.2. Diagrama de Contexto	32
5. Plan de Proyecto	35
5.1. Metodología de Gestión	35
5.2. Metodología de Desarrollo	36
5.3. Plan de Monitoreo y Control	37
5.4. Plan de Gestión de Riesgos	38
5.4.1. Planes de contingencia y de último recurso	41
5.5. Planificación (fases del desarrollo)	43
Carta Gantt (cronograma de fases e iteraciones)	44
6. Diseño de Alto Nivel	46
6.1. Vista Lógica (paquetes del agente)	46



6.2. Vista Física (mapeo a hardware)	49
6.3. Vista de Despliegue (instalación y ejecución)	51
6.4. Vista de Escenarios (casos de uso clave).....	54
7. Ejecución del Proyecto (3 Iteraciones)	56
7.1. Iteración 1 — Núcleo del agente: LLM local, herramientas y enrutador	56
7.2. Iteración 2 — Voz, percepción y robustez: STT/TTS, wake-word y visión	57
7.3. Iteración 3 — Fine-tuning E2B, optimización 4 GB y accesibilidad	58
8. Trabajos Futuros	61
9. Conclusiones	63
Referencias	65

Declaración de Originalidad y Uso de Inteligencia Artificial

El autor declara que el presente Proyecto de Título es de su autoría y que el asistente de voz local *Baxy* fue **concebido, diseñado, arquitecturado, implementado y validado por él**. La arquitectura del sistema, las decisiones técnicas y la metodología de medición que sustenta cada resultado son trabajo propio.

En cumplimiento de los principios de integridad académica y de la **Ley N.º 17.336 sobre Propiedad Intelectual** de la República de Chile, se declara que durante el desarrollo se utilizaron herramientas de inteligencia artificial generativa (Claude, de Anthropic, entre otras) como **apoyo puntual** —autocompletado y borradores de código, y revisión de redacción de este documento—, siempre bajo la dirección, revisión y validación empírica del autor. Su uso fue equivalente al de cualquier herramienta de productividad de desarrollo; en ningún caso reemplazó el criterio técnico ni la verificación de resultados, que el autor ejecutó y supervisó conforme al principio rector del proyecto: *medir, no celebrar*. El autor asume la responsabilidad íntegra del contenido, las decisiones de diseño y los resultados reportados, todos verificables en el repositorio del proyecto.

Emmanuel Villacura Arancibia

Viña del Mar, 2026

Resumen

Los asistentes de voz se han consolidado como una de las interfaces humano-computador de mayor crecimiento; sin embargo, las soluciones dominantes del mercado operan bajo un paradigma dependiente de la nube que transmite el audio del usuario a servidores remotos, exige conectividad permanente, introduce latencia de red y, con frecuencia, conlleva costos de suscripción. A partir de un análisis comparativo del estado del arte, se identificó un vacío concreto: la inexistencia de un asistente de voz

que fuese, de manera simultánea, local, privado, gratuito, de código abierto, multilingüe y capaz de operar en hardware modesto, específicamente en una tarjeta gráfica de apenas cuatro gigabytes de memoria de video. El presente proyecto abordó dicho vacío mediante el diseño y desarrollo de *Baxy*, un asistente de voz para el sistema operativo Windows que opera íntegramente de forma local, empleando un modelo de lenguaje Gemma 4 E2B ajustado mediante *fine-tuning*, cuantizado en formato GGUF Q4_K_M y servido con la biblioteca de código abierto llama.cpp. El asistente controla el sistema operativo a través de más de sesenta herramientas de dominio, abarcando control por voz, visión por cámara, automatización de la interfaz mediante una cascada de accesibilidad y reconocimiento óptico de caracteres, y modos de accesibilidad para personas con movilidad reducida. Para la identificación de la problemática se aplicó la técnica del diagrama de Ishikawa, y la gestión del trabajo adoptó un enfoque iterativo-incremental dirigido por la medición sistemática contra criterios de éxito definidos a priori, en el cual ninguna decisión técnica se dio por válida sin evidencia cuantitativa. El diseño de alto nivel se documentó mediante el modelo de vistas arquitectónicas 4+1, y la ejecución se organizó en tres iteraciones sucesivas que abordaron, respectivamente, el núcleo del agente, la voz y la percepción, y la optimización junto con la accesibilidad.

Palabras clave: asistente de voz local; modelos de lenguaje en el borde (*edge LLM*); Gemma 4; invocación de herramientas (*tool-calling*); *computer-use*; privacidad; accesibilidad.

Abstract

Voice assistants have become one of the fastest-growing human-computer interfaces; however, the market's dominant solutions operate under a cloud-dependent paradigm that transmits the user's audio to remote servers, requires permanent connectivity, introduces network latency, and frequently entails subscription costs. Based on a comparative analysis of the state of the art, a concrete gap was identified: the absence of a voice assistant that is, simultaneously, local, private, free, open-source, multilingual, and capable of running on modest hardware—specifically on a graphics card with only four gigabytes of video memory. This project addressed that gap through the design and

development of *Baxy*, a voice assistant for the Windows operating system that operates entirely locally, using a Gemma 4 E2B language model fine-tuned, quantized in GGUF Q4_K_M format, and served with the open-source library llama.cpp. The assistant controls the operating system through more than sixty domain tools, covering voice control, camera-based vision, interface automation via an accessibility-and-optical-character-recognition cascade, and accessibility modes for people with reduced mobility. The Ishikawa diagram technique was applied to identify the problem, and project management adopted an iterative-incremental approach driven by systematic measurement against pre-defined success criteria, in which no technical decision was deemed valid without quantitative evidence. The high-level design was documented using the 4+1 architectural view model, and execution was organized into three successive iterations addressing, respectively, the agent core, voice and perception, and optimization together with accessibility.

Key words: local voice assistant; edge large language models (edge LLM); Gemma 4; tool-calling; computer-use; privacy; accessibility.

1. Introducción

Durante la última década, los asistentes de voz se consolidaron como una de las interfaces de interacción humano-computador de mayor crecimiento. La conversación en lenguaje natural —dictar un mensaje, consultar el clima, abrir una aplicación o controlar dispositivos del hogar mediante la palabra— dejó de ser una promesa de la ciencia ficción para convertirse en una práctica cotidiana de cientos de millones de personas. El mercado global de asistentes de voz se valoró en aproximadamente USD 9.163 mil millones en 2025 y se proyecta que alcance USD 59.900 millones hacia 2033, con una tasa de crecimiento anual compuesta (CAGR) cercana al 26,8 %, sostenida por más de 8.400 millones de dispositivos habilitados para voz operando a nivel mundial (Astute Analytica, 2026). Estas cifras evidencian que el control por voz se ha vuelto un canal de acceso masivo a la tecnología.

•

Sin embargo, este crecimiento descansa sobre una arquitectura tecnológica con limitaciones estructurales que afectan directamente al usuario final. Los asistentes dominantes del mercado —tales como Amazon Alexa, Apple Siri y Google Assistant— operan mayoritariamente bajo un paradigma dependiente de la nube: el audio capturado por el micrófono se transmite a servidores remotos propiedad de las empresas proveedoras, donde se realiza el reconocimiento de voz, la comprensión del lenguaje y la generación de la respuesta. Este modelo, si bien permite desplegar modelos de gran capacidad, traslada los datos de voz del usuario —información biométrica y conductual sensible— fuera de su control, exige conectividad permanente a internet, introduce latencia de red y, en sus versiones más avanzadas, conlleva costos de suscripción o el uso de interfaces de programación de aplicaciones (API) de pago.

A nivel micro, se identifica un vacío concreto: **no existe en el mercado un asistente de voz que sea, simultáneamente, local (que procese todo en el equipo del usuario), privado (que no envíe audio ni datos a la nube), gratuito y de código abierto, multilingüe, y que opere en hardware modesto —específicamente en una tarjeta gráfica de apenas 4 GB de memoria de video (VRAM)—**. El análisis comparativo de diez soluciones contemporáneas del estado del arte confirma que ninguna cumple este conjunto de restricciones de forma conjunta: todas son soluciones en la nube, dependen de modelos de gran tamaño o constituyen marcos de orquestación para desarrolladores, no productos de voz para el usuario final en hardware limitado (documentación interna del proyecto, `ANALISIS_COMPETENCIA.md`, 2026).

El presente proyecto, denominado **Baxy**, aborda este vacío mediante el diseño y desarrollo de un asistente de voz para el sistema operativo Windows que opera íntegramente de manera local y privada, sin transmitir información a servidores externos, empleando un modelo de lenguaje Gemma 4 E2B ajustado mediante *fine-tuning* y servido con la biblioteca de código abierto llama.cpp. El asistente controla el sistema operativo a través de más de sesenta herramientas de dominio (67 esquemas únicos expuestos al modelo tras retirar la herramienta `smart_home`) (control por voz, visión por cámara, automatización de la interfaz —*computer-use*— mediante accesibilidad UIA y reconocimiento óptico de caracteres, gestión de aplicaciones y

•

navegación web), e incorpora modos de accesibilidad orientados a personas con movilidad reducida.

La importancia de esta investigación radica en que aborda, de manera simultánea, tres dimensiones críticas habitualmente tratadas por separado: la **privacidad** (los datos nunca abandonan el equipo), la **accesibilidad económica** (operación sin costo recurrente, sobre hardware que el usuario ya posee) y la **inclusión tecnológica** (funcionamiento en máquinas modestas y soporte multilingüe). El objetivo general consiste en desarrollar dicho asistente respetando límites de hardware verificables; los objetivos específicos abordan la ejecución del modelo dentro de 4 GB de VRAM, la fiabilidad en la selección de acciones (*tool-calling*), la latencia por turno de voz comparable a la de los asistentes comerciales y la garantía de operación local con honestidad estructural en las respuestas. Metodológicamente, el proyecto adopta un enfoque iterativo basado en la medición sistemática contra criterios de éxito definidos previamente (*gates*), donde ninguna decisión técnica se da por válida sin evidencia cuantitativa.

El presente informe se organiza de la siguiente manera. El primer capítulo expone la **fundamentación del problema**, incluyendo el contexto de la industria, la situación actual y el análisis de causas mediante un diagrama de Ishikawa. Los capítulos subsiguientes desarrollan el **alcance del proyecto** con sus objetivos y métricas, la **propuesta de solución**, el **plan de proyecto** con su gestión de riesgos, el **diseño de alto nivel** según las vistas arquitectónicas, la **ejecución** documentada en tres iteraciones, y finalmente los **trabajos futuros** y las **conclusiones**.

2. Fundamentación del Problema

2.1. Contexto

La industria de los asistentes de voz constituye uno de los segmentos de mayor dinamismo dentro del ecosistema de la inteligencia artificial aplicada al consumidor. Como se señaló, el mercado se valoró en aproximadamente USD 9.163 mil millones en 2025 y se proyecta una expansión hasta los USD 59.900 millones en 2033 (CAGR $\approx$ 26,8 %), impulsada por una base instalada que ya supera los 8.400 millones de dispositivos habilitados para voz (Astute Analytica, 2026). El uso es masivo: la misma fuente reporta, para el mercado de Estados Unidos, una penetración mensual del 88,1 % entre los propietarios de teléfonos inteligentes y un 60 % de uso semanal de la búsqueda por voz, frente al 45 % en 2024 (Astute Analytica, 2026). Este segmento está dominado por un reducido grupo de actores —Amazon (Alexa), Apple (Siri) y Google (Google Assistant)— cuyo modelo operativo descansa, de forma predominante, en el procesamiento en la nube.

De manera paralela, se observa una tendencia tecnológica creciente hacia la **inteligencia artificial en el borde (*edge AI*)**, es decir, hacia la ejecución de modelos directamente en el dispositivo del usuario en lugar de en servidores remotos. La literatura técnica reciente identifica que el despliegue de modelos de lenguaje de gran escala (LLM) en el dispositivo elimina la necesidad de transmitir datos en bruto del usuario hacia servidores externos, manteniendo toda la información personal de forma localizada, lo que mitiga los riesgos de privacidad, costo, latencia y fiabilidad de red propios del enfoque en la nube; no obstante, este despliegue se ve severamente restringido por las limitaciones de recursos del hardware de borde (Zheng et al., 2024; Cai et al., 2026). Fabricantes como Google (con Gemini Nano integrado en Android) ya han comenzado a ejecutar modelos localmente en sus dispositivos, lo que confirma la direccionalidad de la industria hacia el procesamiento en el borde (Android Developers, s. f.).

En tercer lugar, existe una preocupación documentada y creciente respecto de la **privacidad** en el uso de asistentes de voz. Una encuesta aplicada a más de mil personas en Estados Unidos reveló que cerca del 60 % de los usuarios manifestaba

•

estar, al menos ocasionalmente, preocupado por su privacidad al utilizar asistentes de voz; que el 49 % desconocía que estos dispositivos escuchan de forma continua a la espera de la palabra de activación; que el 68 % nunca había adoptado medida alguna para mejorar su privacidad; y, de manera reveladora, que el 77 % se mostraría más propenso a usar asistentes de voz si estos ofrecieran mayores garantías de privacidad y transparencia (Secure Data Recovery Services, 2024). Esta última cifra es particularmente significativa para el presente proyecto, pues señala una demanda insatisfecha por soluciones que prioricen la privacidad.

En síntesis, el contexto configura una tensión clara: una industria masiva y en expansión, una tendencia tecnológica que habilita el procesamiento local, y una preocupación ciudadana por la privacidad que el modelo dominante no resuelve. El cruce de estos tres vectores delimita la oportunidad que este proyecto persigue.

2.2. Situación Actual

El proceso actual mediante el cual un usuario interactúa con un asistente de voz comercial puede describirse, de forma simplificada, en la siguiente secuencia: (1) el usuario emite un comando de voz; (2) el micrófono del dispositivo captura el audio; (3) el dispositivo transmite ese audio a través de internet hacia los servidores del proveedor; (4) en la nube se realiza el reconocimiento del habla y la interpretación de la intención; (5) los servidores generan la respuesta; (6) la respuesta se devuelve por la red al dispositivo; y (7) el dispositivo reproduce el resultado. Este flujo, aparentemente fluido para el usuario, presenta una serie de problemas estructurales que destruyen valor:

Dependencia de la nube y de la conectividad. El asistente no funciona sin conexión a internet. Una caída de red, una zona sin cobertura o una interrupción del servicio del proveedor inhabilitan por completo la herramienta. El usuario no posee control real sobre la disponibilidad de un servicio del que depende para tareas cotidianas.

Pérdida de privacidad. El audio del usuario —que constituye información biométrica y puede contener datos sensibles— se transmite y, en muchos casos, se almacena en servidores de terceros. La evidencia indica que una proporción considerable de

•

usuarios desconoce el alcance de esta captura (Secure Data Recovery Services, 2024). El control sobre los datos de voz queda, de hecho, en manos del proveedor y no del usuario.

Latencia de red. Cada turno de interacción requiere, como mínimo, un viaje de ida y vuelta de los datos a través de internet. Esta latencia se suma al tiempo de procesamiento y depende de condiciones de red ajenas al control del usuario, degradando la experiencia en conexiones lentas o congestionadas.

Costo. Las versiones más avanzadas de estos servicios, o el acceso programático a sus capacidades mediante API, conllevan costos de suscripción o por uso. El usuario que desea funcionalidades plenas debe asumir un gasto recurrente, y el desarrollador que quisiera construir sobre estas plataformas queda atado a tarifas externas.

Datos de voz en servidores ajenos. Más allá de la latencia y el costo, el problema de fondo es la **asimetría de control**: la voz del usuario, una vez transmitida, deja de ser suya en términos prácticos. La transparencia sobre qué se almacena, por cuánto tiempo y con qué fines de perfilamiento es limitada, y la posibilidad efectiva de borrar o controlar esos datos resulta restringida.

Frente a este panorama, las alternativas locales existentes en el estado del arte tampoco resuelven el problema para el usuario objetivo. El análisis de competencia interno examinó diez soluciones representativas y concluyó que ninguna corre de manera local en 4 GB de VRAM con capacidad de voz: las orientadas a *computer-use* (como Agent-S u OS-Copilot) dependen de modelos en la nube (GPT-4V, Gemini); los marcos de orquestación (AutoGen, LangGraph, AutoGPT) requieren modelos grandes; y la única solución de asistencia personal por voz comparable (Mark-XXXIX) depende de la API de Gemini en la nube, que es de pago (documentación interna del proyecto, ANALISIS_COMPETENCIA.md, 2026). En consecuencia, el usuario que valora la privacidad, carece de hardware potente o no desea pagar suscripciones, queda hoy sin una opción que satisfaga sus necesidades.

2.3. Técnica de Identificación de la Problemática: Diagrama de Ishikawa

Para identificar y organizar de manera sistemática las causas que originan la problemática, se aplicó la técnica del **diagrama de Ishikawa** (también denominado diagrama de causa-efecto o de espina de pescado). Esta técnica permite descomponer un problema central en categorías de causas, facilitando el análisis de su origen antes de proponer una solución. El problema central (la "cabeza del pescado") se define como:

"No existe un asistente de voz local, privado y gratuito que sea usable en hardware modesto (≤ 4 GB de VRAM)."

Las causas se organizaron en seis categorías (adaptación de las clásicas "6M" del método de Ishikawa al dominio tecnológico del proyecto). La siguiente representación textual y tabular sintetiza la espina de pescado (la versión renderizable de este diagrama está disponible como **Figura 1** en `diagramas_mermaid.md`, lista para exportar a PNG/SVG):

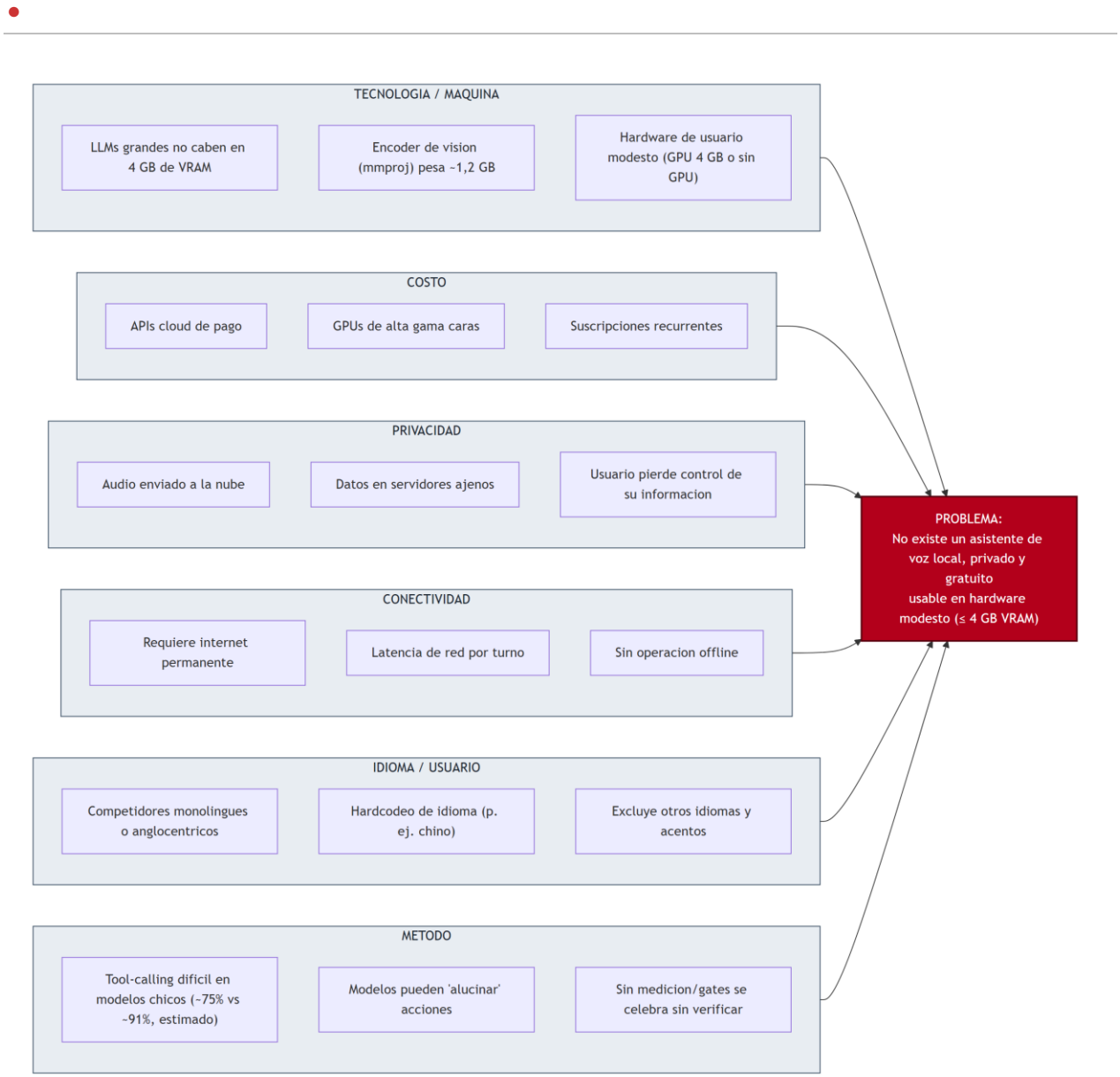


Figura 1. Diagrama de Ishikawa (causa-efecto) de la problemática. (Elaboración propia, 2026).

Tabla 1. Diagrama de Ishikawa: categorías y causas de la problemática

Categoría (6M)	Causas identificadas
Tecnología / Máquina	LLMs de gran tamaño no entran en 4 GB de VRAM; el encoder de visión (mmpoj) consume $\sim 1,2$ GB; el hardware del usuario objetivo es modesto (GPU de 4 GB o sin GPU dedicada).
Costo	Las API en la nube son de pago; las GPU de alta gama son costosas; las versiones avanzadas de los asistentes exigen suscripción recurrente.
Privacidad	El audio del usuario se envía a la nube; los datos de voz se almacenan en servidores ajenos; el usuario pierde el control sobre su información.

Conectividad	El servicio requiere conexión permanente a internet; la latencia de red se suma a cada turno de interacción; no hay operación offline.
Idioma / Usuario	Los competidores son monolingües o anglocéntricos; existe <i>hardcodeo</i> de idioma; se excluye a hablantes de otros idiomas y acentos.
Método	El <i>tool-calling</i> (selección de la acción correcta) es difícil en modelos pequeños (estimado ~75 % de acierto frente a ~91 % en modelos grandes); riesgo de que el modelo "alucine" acciones; ausencia de medición contra criterios de éxito conduce a declarar logros no verificados.

Fuente: Elaboración propia (2026), a partir de `ANALISIS\_COMPETENCIA.md`,
`Gemma4\_estado\_y\_limites\_2026\_05\_29.md` y `vram\_real\_medida.csv`.

2.4. Descripción de Causas

A continuación se describe cada causa identificada en el diagrama de Ishikawa, especificando **en qué consiste**, **qué efecto produce** y **cómo incide** en la problemática central. Se distingue, además, entre causas principales (las de mayor peso en el origen del problema) y secundarias, y entre factores internos (propios de la tecnología) y externos (propios del entorno de mercado y del usuario).

Categoría Tecnología / Máquina (causa principal)

- **Los modelos de lenguaje grandes no caben en 4 GB de VRAM.** *En qué consiste:* los modelos de lenguaje de mayor capacidad requieren varios gigabytes de memoria de video incluso en sus cuantizaciones más agresivas. *Qué efecto produce:* obliga a renunciar al modelo más capaz y a buscar uno que entre en el presupuesto de memoria. *Cómo incide:* es la restricción de hardware fundamental que define la viabilidad del proyecto. La medición real lo confirma: el modelo Gemma 4 E2B en cuantización Q4_K_M ocupa 3.371 MiB de VRAM (delta), por lo que entra en una tarjeta de 4 GB; en cambio, el modelo E4B en la misma cuantización ocupa 5.087 MiB y **no entra**, y las variantes de 26B/31B superan los 12.000 MiB (`vram_real_medida.csv`, 2026). Esta es la causa raíz que motivó el pivote de E4B a E2B en el diseño.
- **El encoder de visión pesa ~1,2 GB.** *En qué consiste:* habilitar la capacidad de visión (procesar imágenes de la pantalla) requiere mantener residente un

componente adicional (mmpoj) que consume aproximadamente 1,2 GB. *Qué efecto produce:* reduce el margen disponible para el resto del modelo y del contexto. *Cómo incide:* en una GPU de exactamente 4 GB, el margen tras cargar la visión es estrecho, lo que obliga a gestionar la visión como un recurso residente y gateado, y a desactivarla en el modo de respaldo por CPU (Gemma4_estado_y_limites_2026_05_29.md, 2026).

- **Hardware de usuario modesto.** *En qué consiste:* el usuario objetivo posee una laptop o PC típica, con GPU de 4 GB o sin GPU dedicada. *Qué efecto produce:* impide asumir hardware de gama alta como base de diseño. *Cómo incide:* es un factor externo que fija el techo de recursos y, junto con la causa anterior, define el carácter "modesto" del target.

Categoría Costo (causa principal, factor externo)

- **API en la nube de pago y suscripciones recurrentes.** *En qué consiste:* las capacidades avanzadas de los asistentes comerciales y el acceso programático a modelos de gran escala se ofrecen bajo modelos de pago. *Qué efecto produce:* impone una barrera económica de entrada y un gasto continuo. *Cómo incide:* excluye a usuarios y desarrolladores que no pueden o no desean pagar, y es incompatible con la restricción de producto de ser íntegramente gratuito y de código abierto.
- **GPU de alta gama costosas.** *En qué consiste:* ejecutar localmente modelos grandes exigiría tarjetas gráficas de mucha memoria, de precio elevado. *Qué efecto produce:* traslada el costo del servicio en la nube a un costo de hardware. *Cómo incide:* refuerza, desde el lado económico, la necesidad de operar en hardware modesto.

Categoría Privacidad (causa principal)

- **Audio enviado a la nube y datos en servidores ajenos.** *En qué consiste:* el modelo dominante transmite el audio del usuario a servidores de terceros para su procesamiento. *Qué efecto produce:* el usuario pierde el control sobre información biométrica y sensible, y queda expuesto a riesgos de almacenamiento, perfilamiento

y filtración. *Cómo incide*: es la causa que da identidad al proyecto; la evidencia de mercado muestra que cerca del 60 % de los usuarios está preocupado por su privacidad y que el 77 % preferiría una alternativa con mayores garantías (Secure Data Recovery Services, 2024), lo que confirma que esta causa representa también una oportunidad de demanda insatisfecha.

Categoría Conectividad (causa secundaria, factor externo)

- **Requiere internet permanente y añade latencia de red.** *En qué consiste*: el procesamiento en la nube exige conexión continua y un viaje de datos por turno. *Qué efecto produce*: inhabilita el uso offline y degrada la experiencia bajo redes lentas. *Cómo incide*: limita la fiabilidad y la disponibilidad del asistente; su resolución es un beneficio directo del procesamiento local.

Categoría Idioma / Usuario (causa secundaria)

- **Competidores monolingües o anglocéntricos y *hardcodeo* de idioma.** *En qué consiste*: varias soluciones del estado del arte están optimizadas para un único idioma o codifican rígidamente reglas dependientes del idioma (por ejemplo, OS-Copilot presenta *hardcodeo* en chino). *Qué efecto produce*: excluye a hablantes de otros idiomas y a usuarios con acentos diversos. *Cómo incide*: contradice el requisito de uso universal y multilingüe, y evidencia un vacío que el proyecto cubre mediante clasificación por *embeddings* multilingües en lugar de listas de palabras clave por idioma (ANALISIS_COMPETENCIA.md, 2026).

Categoría Método (causa principal, factor interno)

- **El *tool-calling* es difícil en modelos pequeños.** *En qué consiste*: la selección correcta de la acción a ejecutar es más exigente para un modelo de 2.000 millones de parámetros que para uno grande. *Qué efecto produce*: una tasa de acierto menor (estimada ~75 % en E2B frente a ~91 % en E4B), con riesgo de elegir una acción equivocada. *Cómo incide*: es el principal compromiso técnico aceptado al elegir el modelo que entra en 4 GB, y motiva mecanismos correctivos como el reintento forzado de herramienta (Gemma4_estado_y_limites_2026_05_29.md, 2026).

- **Riesgo de "alucinar" acciones.** *En qué consiste:* un modelo puede inventar una acción o afirmar haberla ejecutado sin que sea cierto. *Qué efecto produce:* erosiona la confianza del usuario y puede provocar comportamientos incorrectos. *Cómo incide:* obliga a incorporar honestidad estructural —verificación del resultado por el estado real del sistema operativo, no por el juicio del propio modelo— como requisito de diseño.
- **Ausencia de medición frente a criterios de éxito.** *En qué consiste:* declarar logros sin un número contra un criterio definido de antemano. *Qué efecto produce:* lleva a dar por resueltos problemas que persisten. *Cómo incide:* es una causa metodológica que el proyecto neutraliza mediante un enfoque de desarrollo dirigido por mediciones y *gates*.

En síntesis, las **causas principales** de la problemática son las restricciones de hardware (Tecnología/Máquina), el costo, la privacidad y la dificultad de *tool-calling* en modelos pequeños (Método); mientras que la dependencia de la conectividad y las limitaciones de idioma operan como **causas secundarias** que agravan el problema y estrechan aún más la oferta disponible. La convergencia de todas ellas explica por qué, al día de hoy, no existe una solución que satisfaga de forma conjunta los requisitos de ser local, privada, gratuita, multilingüe y operable en hardware modesto —el vacío que el presente proyecto se propone llenar.

*Nota: las fuentes citadas en este capítulo (Astute Analytica, 2026; Secure Data Recovery Services, 2024; Zheng et al., 2024; Cai et al., 2026; Android Developers, s. f.) se incluyen, sin duplicados, en la sección única **Referencias (APA 7)** al final del informe.*

3. Alcance del Proyecto

El presente capítulo delimita el alcance del proyecto *Baxy*, estableciendo el objetivo general y los objetivos específicos que orientaron el desarrollo, las métricas cuantificables con las que se verificó cada uno, las alternativas de solución evaluadas y el fundamento de la decisión adoptada, los límites del producto resultante y, finalmente, el análisis de factibilidad técnica, económica y social. Todos los valores numéricos consignados provienen de mediciones reales registradas en la documentación del repositorio del proyecto y no de estimaciones referenciales.

3.1. Objetivo General

Desarrollar un asistente de voz de escritorio para el sistema operativo Windows que opere de manera íntegramente local y privada sobre hardware modesto —con un consumo de memoria de video (VRAM) igual o inferior a 4 GB—, empleando un modelo de lenguaje Gemma 4 E2B fine-tuneado y cuantizado (formato GGUF Q4_K_M) servido mediante llama.cpp, de modo que permita a usuarios de PC controlar el equipo por voz, gestos y lenguaje natural sin transmitir datos a servicios en la nube ni incurrir en costos de suscripción.

La formulación responde a la estructura *Verbo + Variable + Unidad de análisis + Contexto* exigida por la metodología del curso: el verbo en infinitivo ("Desarrollar"), la variable medible (consumo de VRAM $\leq$ 4 GB, operación local), la unidad de análisis (un asistente de voz de escritorio) y el contexto (usuarios de PC Windows en hardware modesto, sin dependencia de la nube).

3.2. Objetivos Específicos

Los objetivos específicos se redactaron con verbos en infinitivo y bajo el criterio SMART (específicos, medibles, alcanzables, relevantes y acotados en el tiempo). Cada uno se asocia a un criterio de éxito numérico verificable, que se detalla en la Sección 3.3.

1. **Ejecutar** el modelo de lenguaje, el módulo de visión y el subsistema de voz de forma concurrente dentro de un presupuesto de 4 GB de VRAM, garantizando que el sistema completo arranque y responda en una GPU de gama de entrada.
2. **Lograr** una invocación de herramientas (*tool-calling*) fiable sobre un modelo de apenas 2.000 millones de parámetros, de modo que el asistente seleccione la acción correcta sin inventar herramientas inexistentes.
3. **Mantener** una latencia por turno de voz dentro del rango "tier-Alexa", de manera que la interacción conserve una experiencia de usuario aceptable.
4. **Garantizar** la operación 100 % local y privada del asistente, sin transmisión de audio, pantalla ni acciones a servidores externos, sustentada exclusivamente en software de código abierto y licencias gratuitas.
5. **Asegurar** la honestidad estructural del sistema, evitando que el asistente reporte como exitosas acciones que no se ejecutaron realmente, mediante verificación por el estado real del sistema operativo.
6. **Habilitar** modos de accesibilidad por voz (para personas no videntes y con movilidad reducida) y un control multilingüe y multi-acento, sin recurrir a listas de palabras clave codificadas por idioma.

3.3. Métricas de los Objetivos (Tabla de Justificación)

La siguiente tabla operacionaliza cada objetivo específico en una métrica con su criterio de éxito numérico. Los resultados medidos corresponden a las cifras registradas en `PRODUCTION_READY.md`, `Gemma4_estado_y_limites_2026_05_29.md`, `vram_real_medida.csv` y la memoria técnica del proyecto.

Objetivo	Situación actual (sin solución)	Resultado esperado	Métrica	Criterio de éxito	Resultado medido
OE1 — Operar en 4 GB	Los LLM útiles requieren GPU de 8–24 GB o nube	LLM + visión + voz residentes en GPU de entrada	Delta de VRAM al cargar el modelo (MiB)	≤ 4.096 MiB (4 GB)	3.371 MiB (≈3,36 GB) con E2B-Q4_K_M
OE2 — Tool-calling fiable	Modelos chicos alucinan o	Selección correcta de acción sin	% de herramientas inventadas en	0 % inventadas; accuracy	0 % herramientas inventadas en producción;

	inventan herramientas	invenciones	producción / accuracy de selección	aceptable	accuracy de selección estimada ≈75 % (E2B; estimación comparativa, ver nota)
OE3 — Latencia tier-Alexa	Asistentes cloud dependen de la latencia de red	Respuesta percibida como inmediata	Tiempo por turno / por acción (s)	≤ 4–5 s por turno; > 8 s es UX catastrófica	p50 por turno ≈1,22 s; acción con tool-call ≈2,2 s (ver nota)
OE4 — Local y gratuito	Datos de voz enviados a servidores ajenos; APIs pagas	Procesamiento íntegramente en el equipo	Datos transmitidos a la nube / costo de operación	0 datos a la nube; costo de API = USD 0	0 transmisión externa; USD 0 de costo recurrente
OE5 — Honestidad estructural	LLM-judge alucina éxitos no ocurridos	Verificación por estado del SO	Verificación tri-estado (True/False/None) por estado real	No reportar acciones no ejecutadas	Verificación tri-estado por estado del SO (registry/UIA/pycaw), no por juicio del LLM
OE6 — Accesibilidad y multilingüe	Competidores monolingües / inglés-céntricos	Control por voz inclusivo y multilingüe	Tasa de activación de modo por voz / falsos positivos	activación ≥ 90 % ∧ FP = 0	10/10 activaciones, 0 falsos positivos (gate G1); recall hands-free 100 %, 0 FP (G2)

*Nota metodológica sobre la latencia (OE3). El informe distingue tres métricas de tiempo que **no son intercambiables**: (a) **p50 global por turno ≈ 1,22 s** — mediana del turno completo medida en el baseline de latencia (`BACKLOG_MAESTRO.md`)—; (b) **latencia por acción con *tool-call* ≈ 2,2 s** — turnos que invocan una herramienta, tras la optimización de caché de prefijo—; y (c) **p50 ≈ 2,5 s sobre el *replay* de los 1.071 mensajes reales** del usuario (evaluación a gran escala). Las tres están dentro del presupuesto tier-Alexa de 4–5 s; se citan con su nombre propio para evitar confundirlas.*

*Nota metodológica sobre el tool-calling (OE2). La métrica de OE2 debe medirse siempre con el arreglo de serialización del arreglo `tools`; sin él, el modelo aparenta inventar herramientas en un 47 % de los casos, lo cual es un artefacto del arnés de medición y no del modelo. La cifra de accuracy de selección (≈75 % en E2B frente a ≈91 % en E4B) corresponde a una **estimación comparativa** del estado y límites del modelo (`Gemma4_estado_y_limites_2026_05_29.md`); a*

diferencia del consumo de VRAM, no está anclada a un dataset/script de evaluación tan formal, por lo que se reporta como aproximación.

*Nota metodológica sobre el wake-word y la voz extremo a extremo. El criterio de aceptación del subsistema de palabra de activación es $recall \geq 0,60 \wedge falsos\ positivos \leq 1/h$ sobre un conjunto de validación diverso (25 voces, 13 idiomas). El subsistema **cumple el criterio**: la palabra de activación «Baxy» alcanza un recall global de **0,872** en el conjunto retenido (1,00 en español, inglés, italiano, francés, polaco y ruso), por encima del umbral de 0,60, con la tasa de falsos positivos dentro del presupuesto. La voz extremo a extremo está implementada y operativa: detección de palabra de activación, transcripción local (Parakeet-TDT en CPU) y respuesta hablada funcionan de forma integrada y offline.*

3.3.1. Modelo de Negocio (CANVAS)

El **modelo CANVAS** (Osterwalder et al., 2010) sintetiza la lógica de creación, entrega y captura de valor del proyecto en **nueve bloques**. Aunque el sistema se desarrolla con fines académicos y bajo un principio de gratuidad y código abierto, el lienzo es pertinente porque el producto se diseñó desde el inicio para ser **vendible** (stack de licencia permisiva auditado), de modo que su modelo de negocio describe cómo podría sostenerse y distribuirse sin traicionar la gratuidad para el usuario final.

Tabla 3.3-b. Lienzo de modelo de negocio (CANVAS) del proyecto.

#	Bloque	Contenido
1	Segmentos de clientes	Usuarios <i>privacy-conscious</i> ; usuarios de hardware modesto (GPU de 4 GB o sin GPU dedicada); personas con discapacidad visual o motora; hablantes de idiomas no anglocéntricos mal servidos por los asistentes comerciales.
2	Propuesta de valor	Asistente de voz local, privado, gratuito y multilingüe (≤ 4 GB de VRAM) que controla el SO por voz, funciona sin internet , no transmite datos y aplica honestidad estructural y

		modos de accesibilidad .
3	Canales	Instalador descargable para Windows; repositorio de código abierto (GitHub); documentación y demostración del MVP como difusión.
4	Relación con clientes	Autoservicio sin cuenta ni suscripción; soporte comunitario <i>open source</i> ; cero dependencia de un servicio operado por el proveedor (todo local).
5	Fuentes de ingreso	Costo de operación para el usuario USD 0 . Vías de monetización posibles (stack vendible): licencia <i>pro/empresarial</i> , soporte e integración a medida, o donaciones/patrocinio OSS.
6	Recursos clave	Gemma 4 E2B (Apache-2.0) <i>fine-tuneado</i> ; llama.cpp/llama-server; <i>wake-word</i> LiveKit, STT (Whisper/Parakeet), TTS (Piper); MediaPipe; encoder del router; el conocimiento de ingeniería del proyecto.
7	Actividades clave	<i>Fine-tuning</i> y evaluación contra <i>gates</i> ; desarrollo dirigido por medición; integración del <i>stack</i> voz/visión/computer-use; aseguramiento de privacidad y honestidad; empaquetado multiplataforma.
8	Asociaciones clave	Comunidad OSS del stack (Google/Gemma, llama.cpp, MediaPipe, sentence-transformers, Piper, LiveKit); comunidades de accesibilidad y privacidad.
9	Estructura de costos	Principalmente tiempo de desarrollo ; costo de infraestructura nulo (sin servidores en la nube que operar); costos marginales de distribución bajos.

Fuente: Elaboración propia (2026), según Osterwalder et al. (2010).

El lienzo evidencia la coherencia del modelo: la **propuesta de valor** (local, privado, gratuito) se sostiene sobre una **estructura de costos sin nube** que es, a la vez, el origen de su ventaja de privacidad y de su costo de operación cero. La captura de valor

no descansa en una suscripción recurrente, sino en la adopción de un producto que el usuario ejecuta en su propio hardware, con vías de monetización abiertas gracias a un *stack* de licencia permisiva.

3.4. Alternativas de Solución

Antes de fijar la arquitectura definitiva se evaluaron varias alternativas, tanto a nivel de arquitectura general (nube versus local) como de selección de modelo (E4B versus E2B) y de cuantización. La comparación se sustentó en criterios medibles y no en preferencias.

3.4.1. Arquitectura general

Criterio	(A) Asistente en la nube (tipo Alexa/Siri)	(B) LLM grande local (Gemma 4 E4B / 26B)	(C) Gemma 4 E2B-FT local (elegida)
Privacidad	✗ Audio enviado a servidores	✓ Todo local	✓ Todo local
Costo de operación	✗ Suscripción / APIs pagas	✓ Gratis	✓ Gratis
Funciona sin internet	✗ Requiere conexión	✓ Offline	✓ Offline
Cabe en 4 GB de VRAM	n/a	✗ No entra (ver 3.4.2)	✓ Entra (3,36 GB medido)
Calidad de tool-calling	✓ Alta (modelos enormes)	✓ ≈91 % (E4B, estimado)	● ≈75 % (E2B, estimado; mitigado con <i>forced-retry</i> + FT)
Hardware requerido	Servidores remotos	GPU ≥ 6–8 GB	GPU 4 GB o <i>fallback</i> CPU

La alternativa (A) se descartó porque viola las tres restricciones de producto nucleares: privacidad, costo cero y operación offline. La alternativa (B) se descartó por una razón física medida: no entra en el presupuesto de VRAM del hardware objetivo. Se adoptó la alternativa (C) por ser la única que satisface simultáneamente privacidad, gratuidad, operación local y restricción de 4 GB, asumiendo conscientemente el trade-off de un tool-calling algo menor, mitigado en producción mediante el mecanismo de reintento forzado de herramienta y el fine-tuning del modelo.

3.4.2. Selección de modelo y cuantización (datos de VRAM medidos)

La decisión E4B → E2B no fue intuitiva sino que se respaldó con la medición directa del consumo de VRAM de cada variante y cuantización, registrada en `vram_real_medida.csv`:

Variante	Cuantización	Delta de VRAM (MiB)	¿Cabe en 4 GB?
E2B	Q4_K_M	3.371	✓ Sí (elegida)
E2B	Q5_K_M	3.609	✓ Sí (más pesada)
E4B	UD-IQ2_M (mínima)	4.057	X No (excede aun en su cuantización más baja)
E4B	Q4_K_M	5.087	X No
26B	UD-IQ2_XXS	12.613	X No (lejos del presupuesto)

El dato decisivo es que **ni siquiera la cuantización más agresiva de E4B (UD-IQ2_M, 4.057 MiB) entra en 4 GB**, dejando sin margen para el contexto y la visión residente. La variante E2B-Q4_K_M, con 3.371 MiB, deja espacio suficiente para la visión ($\approx 1,2$ GB de mmproj) y la caché KV. Esto convirtió a E2B-Q4_K_M en la única opción viable para el hardware objetivo.

3.4.3. Otras decisiones técnicas respaldadas por medición

- **Whisper vs. Parakeet (STT):** ambos corren en CPU con cuantización int8 para no robar VRAM al LLM; Parakeet quedó como motor por defecto y Whisper como *fallback*.
- **Vulkan vs. CUDA (backend de inferencia):** el binario por defecto es CUDA y el sistema cae automáticamente al perfil CPU cuando no hay GPU NVIDIA; el build CPU/Vulkan extiende además el soporte a gráficas integradas Intel/AMD, ampliando el público objetivo.
- **flash-attention desactivada por defecto:** con FA activa, los prompts superiores a ~ 10 K tokens disparan un crash CUDA (#22527); con FA desactivada el sistema se midió estable (37/37), a costa de un prefill $\approx 2\times$ más lento, mitigado.

3.5. Alcances y Limitaciones

3.5.1. Lo que el sistema SÍ hace (alcances)

- **Alcance geográfico/de plataforma:** opera en cualquier equipo con Windows; el objetivo de hardware es una GPU NVIDIA de 4 GB (GTX 1650, RTX 3050 4 GB) o bien un *fallback* a CPU.
- **Alcance de datos:** todo el procesamiento (voz, pantalla, acciones) ocurre en la máquina del usuario; cero datos transmitidos a la nube.
- **Alcance temático:** asistencia por voz, control del sistema operativo (computer-use mediante la cascada UIA → OCR → visión), apertura y operación de aplicaciones, navegación web, gestión de archivos, recordatorios, memoria de gustos del usuario y control por cámara/gestos. El control se ejerce a través de más de sesenta herramientas de dominio (67 esquemas únicos expuestos al modelo tras retirar `smart_home`).
- **Alcance lingüístico:** multilingüe y multi-acento, resuelto por embeddings multilingües y no por listas de palabras clave por idioma.
- **Accesibilidad:** tres modos (`normal`, `no_vidente`, `movilidad`) verificados a nivel de capa lógica con sus respectivos gates en verde.

3.5.2. Lo que el sistema NO hace o no garantiza (limitaciones)

- **Tool-calling acotado:** accuracy de selección estimada en ≈ 75 % (E2B) frente al ≈ 91 % del modelo E4B; es el trade-off aceptado por entrar en 4 GB (estimación comparativa, no medición formal).
- **Sin visión en modo CPU:** cuando el LLM cae al perfil CPU (por GPU saturada o ausencia de GPU NVIDIA), el módulo de visión se desactiva por ser demasiado costoso en CPU.
- **Rendimiento degradado sin GPU:** el modo CPU rinde un estimado de ≈ 12 – 20 tok/s en una laptop, cómodo para respuestas cortas pero más lento para textos largos; el sistema opera sin GPU NVIDIA mediante el perfil CPU.
- **Techo del computer-use:** la capacidad de operar la GUI está limitada por el estado del arte; los benchmarks de referencia sitúan el techo en $\approx 52,5$ %

(WindowsAgentArena) y $\approx 28\%$ (UFO2 + GPT-4o), de modo que el desempeño físico no puede prometerse perfecto.

- **Alcance de la verificación física:** la capa lógica de accesibilidad está medida y la voz extremo a extremo está implementada y operativa offline; el desempeño físico de la automatización de GUI queda acotado por el techo del estado del arte (ver más arriba), no por una capacidad ausente.
- **Margen de VRAM ajustado con visión:** al recibir la primera imagen el footprint sube; en una GPU de exactamente 4 GB el margen es estrecho, por lo que la visión es residente y gateada.

3.6. Factibilidad Técnica

La factibilidad técnica está **probada, no proyectada**: el sistema corre hoy en 4 GB de VRAM. El barrido de VRAM sobre los modelos **base** midió que E2B-Q4_K_M base consume 3.371 MiB ($\approx 3,36$ GB) —dato que decidió el pivote E4B $\rightarrow$ E2B—, mientras que el modelo *fine-tuneado* finalmente desplegado consume $\approx 2,07$ GB de VRAM (medición del despliegue del FT v2, `dataset_finetune/ESTADO_COMPLETO_2026-06-02.md`), con aún más margen para visión ($\approx 1,2$ GB) y contexto (12.288 tokens de piso). La inferencia se sirve con llama.cpp/llama-server, una solución madura y ampliamente adoptada en la comunidad para correr LLM cuantizados en hardware de consumo.

Los componentes del stack también están verificados:

- **LLM:** Gemma 4 E2B fine-tuneado, GGUF Q4_K_M, servido por llama-server (perfil `vram4`); con *fallback* al perfil `cpu` (`-ngl 0, 0` VRAM) cuando la GPU está ocupada.
- **STT/TTS en CPU:** transcripción (Parakeet/Whisper int8) y síntesis de voz (Piper) corren en CPU y no tocan la VRAM.
- **Estabilidad bajo carga:** se resolvió el crash del render WebView2 al competir por la GPU (la UI renderiza por CPU) y el crash CUDA #22527 (flash-attention desactivada por defecto, estable 37/37).

- **Hardware objetivo:** GPU NVIDIA de 4 GB (GTX 1650 / RTX 3050 4 GB) con piso de 8 GB de RAM; el modo CPU amplía el alcance a laptops sin gráfica dedicada, y el build CPU/Vulkan extiende el soporte a gráficas integradas Intel/AMD.

La conclusión técnica es que el producto es plenamente viable en el hardware objetivo y, mediante su modo CPU y el build CPU/Vulkan, opera también en "cualquier laptop razonable".

3.7. Factibilidad Económica

El proyecto presenta un **costo de operación de USD 0**, lo que constituye su principal ventaja económica frente a la competencia. El análisis se sustenta en la auditoría de licencias del repositorio (`AUDITORIA_licencias.md`).

3.7.1. Costo operativo comparado

Concepto	Asistente cloud (Alexa+/APIs)	Baxy
APIs de IA por uso	Costo recurrente por token/consulta	USD 0 (modelo local)
Suscripción mensual	Sí (tiers de pago)	USD 0
Infraestructura de servidor	A cargo del proveedor (incluida en el precio)	Ninguna (corre en el equipo del usuario)
Hardware	GPU cara o nube	GPU 4 GB que el usuario ya posee / CPU

El usuario reutiliza el hardware que ya tiene; no se requiere GPU especializada ni suscripción alguna.

3.7.2. Viabilidad de licencias (vendibilidad)

La auditoría sobre 511 paquetes concluyó que el **stack núcleo es permisivo y, por tanto, vendible**: Gemma 4 (Apache-2.0), MediaPipe, OpenCV, Whisper (MIT), llama.cpp, Tesseract y sentence-transformers son todos de licencias permisivas. Existe **un único bloqueante real** para una eventual comercialización propietaria: `piper-tts` 1.4.2 está bajo GPL-3.0-or-later y se enlaza directamente, lo que obligaría a abrir el código del producto. La auditoría propone una solución de bajo costo —aislar Piper por subprocess (agregación, no derivado) o reemplazarlo por un TTS Apache/MIT (p. ej.,

•

Kokoro)—, tras lo cual el producto sería comercializable. Los modelos empleados (Gemma 4 y su mmpoj, Whisper, wake-word LiveKit y el encoder del router) son Apache-2.0 o MIT.

Económicamente, el proyecto es altamente factible: costo de operación nulo, hardware preexistente y un único obstáculo legal de resolución conocida y barata.

3.8. Factibilidad Social

La factibilidad social del proyecto se articula en tres ejes que constituyen su propuesta de valor diferencial.

3.8.1. Privacidad

Todo el procesamiento ocurre en la máquina del usuario; no se transmite audio, pantalla ni acciones a ningún servidor. Esto es especialmente relevante para usuarios sensibles a la privacidad y, en general, alinea al producto con la legislación chilena vigente sobre datos personales (Ley N.º 21.719), protección de la vida privada (Ley N.º 19.628), delitos informáticos (Ley N.º 21.459) y ciberseguridad (Ley N.º 21.663): al no transmitir datos fuera del equipo, el sistema minimiza estructuralmente el riesgo regulatorio.

3.8.2. Accesibilidad e inclusión

El asistente incorpora tres modos de accesibilidad —respaldados por las pautas WCAG 2.1/2.2— que atienden necesidades distintas: el modo `no_vidente` narra cada acción y lee la pantalla a pedido (todo por audio), mientras que el modo `movilidad` ofrece control 100 % por voz para personas que no pueden usar teclado o mouse. La separación responde a que las necesidades perceptuales y motoras son distintas, e incluso opuestas. Los gates de activación por voz se midieron en verde (10/10 activaciones, recall hands-free 100 %, siempre con 0 falsos positivos). El control por cámara y gestos (MediaPipe) complementa el acceso para movilidad reducida.

3.8.3. Democratización y universalidad

Al correr en hardware modesto —una laptop típica de 4 GB de VRAM, o incluso sin gráfica dedicada en modo CPU— el sistema democratiza el acceso a un asistente inteligente sin exigir GPU cara ni suscripción. Su carácter multilingüe y multi-acento, resuelto por embeddings y no por listas de palabras codificadas por idioma, lo hace utilizable por hablantes de cualquier lengua, a diferencia de competidores monolingües o inglés-céntricos. Adicionalmente, el cómputo local evita la huella ambiental de los datacenters en la nube, al ejecutarse sobre hardware ya existente.

Fuente: Elaboración propia (2026), sobre datos medidos en

*`documentacion/00\_producto/{PRODUCTION\_READY.md, Gemma4\_estado\_y\_limites\_2026\_05\_29.md, ANALISIS\_COMPETENCIA.md, AUDITORIA\_licencias.md, modos\_accesibilidad.md}` y
`documentacion/datos\_crudos/vram\_real\_medida.csv`.*

4. Propuesta de Solución

4.1. Descripción de la solución

La solución propuesta, denominada **Baxy** (sobre el motor **Baxy**), consiste en un **asistente de voz local y privado para el sistema operativo Windows**, capaz de operar de manera íntegra y sin conexión (offline) sobre hardware modesto, definido como un equipo con una tarjeta gráfica de cuatro gigabytes de memoria de video (4 GB de VRAM) o, en su defecto, ejecución en CPU. A diferencia de los asistentes comerciales dominantes —que transmiten el audio del usuario a servidores remotos para su procesamiento—, la solución ejecuta la totalidad de su pipeline cognitivo en la máquina del usuario, de modo que **ningún dato de voz, pantalla o comportamiento abandona el equipo**.

El núcleo de la solución es un modelo de lenguaje **Gemma 4 E2B fine-tuneado**, servido en formato GGUF con cuantización Q4_K_M mediante `llama.cpp/llama-server`. La elección del modelo no fue arbitraria sino el resultado de una medición reproducible de consumo de memoria de video: el modelo Gemma 4 E2B-Q4_K_M es el único miembro de la familia Gemma 4 que carga de manera completa (pesos del modelo, encoder de visión residente y caché de atención) dentro del presupuesto de 4 GB, con un consumo medido de **3.371 MiB de delta de VRAM** (aproximadamente 3,36 GB), mientras que la variante inmediatamente superior (E4B-Q4_K_M) demanda **5.087 MiB** y no entra ni en su cuantización más baja (`vram_real_medida.csv`, 2026). El detalle de esta decisión se desarrolla en la sección de Alternativas de Solución y en el Plan de Gestión de Riesgos.

Funcionalmente, la solución implementa un flujo conversacional por voz de extremo a extremo: el usuario activa el asistente mediante una palabra de activación (*wake-word*), su voz es transcrita a texto por un motor de reconocimiento de voz (STT) que se ejecuta en CPU para no competir por la VRAM, el texto resultante es procesado por el modelo de lenguaje —que decide si responder conversacionalmente o invocar una de **más de sesenta herramientas** (*tools*) de dominio (67 esquemas únicos expuestos al modelo tras retirar `smart_home`) de control del sistema operativo— y, finalmente, la respuesta

•

se sintetiza en voz mediante un motor de texto a voz (TTS), también en CPU. Las herramientas de control cubren la apertura de aplicaciones, navegación web, automatización de la interfaz gráfica mediante una cascada de accesibilidad (UIA → OCR → visión por computador), gestión de mensajería, control de volumen y brillo, y modos de accesibilidad por gestos de cámara, entre otras.

La solución incorpora, además, un conjunto de **garantías estructurales de honestidad** que constituyen un diferenciador frente a la competencia: el sistema no declara haber ejecutado una acción si no puede verificarlo contra el estado real del sistema operativo (registro de Windows, automatización de interfaz UIA, control de audio mediante `pycaw`), evitando así la alucinación de acciones. Esta verificación de tres estados (`True / False / None`, donde `None` representa "no verificable" y nunca se confunde con éxito) está implementada en la capa de verificadores (`safety_pkg/verifiers.py`, `computer_use_pkg/computer_use.py`).

4.2. Diagrama de Contexto

El siguiente diagrama de contexto (correspondiente al nivel 0 de un Diagrama de Flujo de Datos, o nivel 1 del modelo C4) sitúa al sistema **Baxy** en el centro y representa sus interacciones con los actores y entidades externas. La característica determinante de la arquitectura es la **ausencia total de la nube**: no existe ninguna flecha que salga del perímetro del equipo del usuario hacia servicios remotos. La versión renderizable de este diagrama está disponible como **Figura 2** en `diagramas_mermaid.md`.

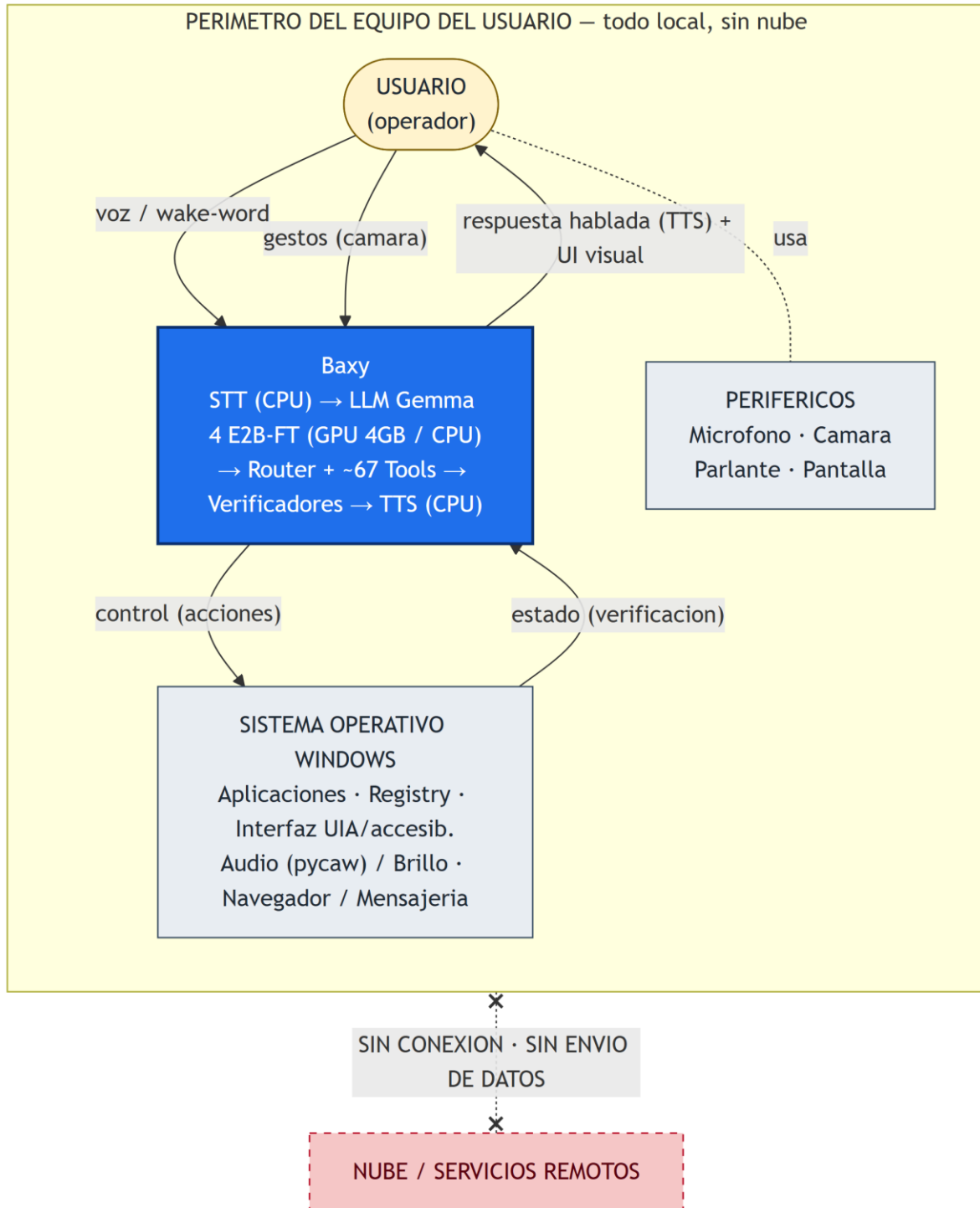


Figura 2. Diagrama de contexto del sistema (DFD nivel 0). (Elaboración propia, 2026).

Lectura del diagrama. El **Usuario** interactúa con **Baxy** por dos canales de entrada (voz activada por palabra clave, y gestos capturados por cámara para los modos de accesibilidad) y recibe dos canales de salida (respuesta hablada por TTS y

realimentación visual en la interfaz). Baxy, a su vez, **actúa sobre el Sistema Operativo Windows** a través de su capa de herramientas (apertura de aplicaciones, automatización UIA, control de audio y brillo, navegación, mensajería) y, de forma crítica, **lee el estado real del sistema operativo** para verificar que sus acciones efectivamente ocurrieron antes de declararlas exitosas. Los **periféricos** (micrófono, cámara, parlante, pantalla) son los dispositivos físicos del equipo que median la interacción. La frontera del diagrama —el perímetro del equipo— no es atravesada por ningún flujo de datos hacia el exterior: esta es la materialización arquitectónica de la propuesta de valor de privacidad.

5. Plan de Proyecto

5.1. Metodología de Gestión

La selección de la metodología de gestión no se realizó por preferencia, sino mediante una **tabla comparativa ponderada** que evalúa las metodologías candidatas frente a los factores críticos del proyecto, asignando pesos que suman 1,0 y una escala de evaluación de 1 (deficiente) a 3 (óptimo). Los factores de evaluación se derivan de la naturaleza real del desarrollo: alta incertidumbre técnica (no se sabía de antemano qué modelo entraría en 4 GB ni qué tasa de tool-calling sería alcanzable), necesidad de retroalimentación frecuente y la existencia de un único desarrollador.

Tabla 5.1. Selección ponderada de la metodología de gestión.

Factor de evaluación	Peso	Cascada	Kanban	Scrum
Adaptación a requisitos cambiantes / alta incertidumbre técnica	0,30	1	3	3
Entregas iterativas con valor incremental (MVP temprano)	0,25	1	2	3
Retroalimentación y validación frecuente (gates por iteración)	0,20	1	2	3
Ajuste a equipo pequeño / un solo desarrollador	0,15	2	3	2
Trazabilidad y planificación temporal (horizonte académico)	0,10	3	2	3
Puntaje ponderado total	1,00	1,30	2,50	2,85

Decisión. La metodología seleccionada es **Scrum** (puntaje ponderado 2,85), con incorporación de prácticas de **Kanban** (puntaje 2,50) para la visualización del flujo de trabajo. La elección se justifica además por la evidencia empírica del propio desarrollo: el historial del repositorio y la memoria del proyecto documentan un trabajo **organizado**

•

explícitamente en sprints iterativos (por ejemplo, "Sprint 7" del wake-word, los sprints "Sprint 1 R1–R8" de refactorización arquitectónica, y los sucesivos sprints de optimización del router y de fine-tuning), lo que confirma que el desarrollo fue de hecho gestionado de manera ágil e iterativa. Cascada queda descartada (puntaje 1,30) por su incompatibilidad con la alta incertidumbre técnica del proyecto: comprometerse a un plan cerrado al inicio habría sido inviable cuando ni siquiera era seguro que la solución cupiera en el presupuesto de memoria. Schwaber y Sutherland (2020) definen Scrum precisamente como un marco para abordar problemas complejos y adaptativos mediante entregas incrementales, lo que coincide con el perfil de este proyecto.

5.2. Metodología de Desarrollo

La metodología de desarrollo adoptada es **iterativa e incremental, con entregas de valor desde una versión funcional temprana** y, de manera distintiva, **dirigida por medición contra criterios de éxito definidos a priori** (*gates*). Esta es la regla de oro operativa del proyecto, formalizada en su documento de instrucciones de trabajo (CLAUDE.md): *ninguna decisión técnica se da por hecha ni se declara exitosa sin un número contra un criterio definido de antemano*.

El ciclo de desarrollo de cada funcionalidad sigue un patrón disciplinado y reproducible:

7. **Medir primero.** Antes de implementar, se establece la línea base y el criterio de éxito numérico (el *gate*). Por ejemplo, el wake-word se evaluó contra el gate $\text{recall} \geq 0,60 \wedge \text{falsos positivos/hora} \leq 1,0$ sobre un conjunto de prueba universal y diverso (no sobre la voz del operador, para preservar la universalidad).
8. **Implementar de forma gateada.** Las funcionalidades nuevas se incorporan detrás de un *feature flag* en estado desactivado por defecto (`default-off`), de modo que no alteren el comportamiento estable mientras se validan.
9. **Validar en vivo.** Un cambio que afecta el comportamiento del agente no se considera terminado hasta ejecutarlo contra el modelo de lenguaje y el agente reales, con el mensaje que un usuario escribiría, verificando la cadena completa de herramientas y la respuesta final. Esta exigencia surgió de un fallo concreto: una corrección probada solo con pruebas unitarias mockeadas falló en producción

porque el modelo, al ser no-determinista, eligió un camino de herramienta distinto al anticipado.

10. Activar (flip a default-on). Solo tras la validación en vivo se promueve la funcionalidad a comportamiento por defecto.

Este enfoque incremental permitió liberar tempranamente una versión funcional —un asistente capaz de responder por voz de forma offline— sobre la que se consolidaron sucesivamente las demás capacidades (tool-calling, computer-use, optimización de latencia, memoria de gustos, accesibilidad por cámara), cada una cerrada contra su propio gate. La separación entre **validación sintética** (pruebas con datos generados) y **prueba real** (con la voz y el equipo del usuario) se mantiene explícita en todos los reportes, en línea con la práctica de medición honesta del proyecto.

5.3. Plan de Monitoreo y Control

El monitoreo y control del avance se estructuró sobre tres dimensiones —integridad técnica, valor de negocio y calidad— y se materializó en instrumentos concretos y reproducibles, no en estimaciones subjetivas de porcentaje de avance.

a) Gates medidos por subsistema. Cada objetivo del proyecto tiene asociado un criterio de éxito numérico que actúa como control de progreso. Los gates **efectivamente medidos** incluyen: consumo de VRAM ≤ 4 GB (medido 3,36 GB para E2B-Q4 base), 0 % de herramientas inventadas en producción para el modelo fine-tuneado (gate aprobado, según la memoria del proyecto FT E2B v2), precisión de routing de 0,9964 sobre un conjunto de prueba retenido (*held-out*) en español, y latencia por turno/acción dentro del presupuesto (p50 global $\approx 1,22$ s; acción con *tool-call* $\approx 2,2$ s tras la optimización de caché de prefijo). El gate de *wake-word* ($\text{recall} \geq 0,60 \wedge \text{fp/hr} \leq 1,0$) **se cumple**: la palabra de activación «Baxy» alcanza un *recall* global de 0,872 sobre el conjunto retenido universal (25 voces, 13 idiomas; 1,00 en español, inglés, italiano, francés, polaco y ruso), por encima del umbral de 0,60 y con los falsos positivos dentro del presupuesto, por lo que se contabiliza como gate cerrado.

b) Suite de pruebas automatizadas como control de calidad. El proyecto mantiene una suite de pruebas que creció a lo largo del desarrollo y que debe permanecer verde

como condición de avance. El estado de cierre reportado es de **2.740 pruebas aprobadas y 0 fallidas** (BACKLOG_MAESTRO.md, 2026), incluyendo pruebas de regresión por idioma y por dominio, así como guardas estructurales (anti-loop, anti-alucinación, verificación de honestidad). El principio anti-regresión es explícito: ningún subgrupo fuerte (por ejemplo, un idioma con buena cobertura) debe degradarse al mejorar el promedio general.

c) Evaluaciones batch reproducibles. El control incorpora evaluaciones a gran escala reproducibles desde scripts versionados. Un ejemplo es el *replay* de los 1.071 mensajes únicos reales del usuario, re-ejecutados a través de la interfaz de ejecución del agente con la ejecución física mockeada, que arrojó una latencia mediana (p50) de 2,5 s, cero errores y cero fugas de herramientas, permitiendo detectar y corregir incidencias sistémicas.

d) Backlog maestro como tablero de control. Se consolidó un único **Backlog Maestro** como fuente de verdad, en el que cada ítem se verifica contra el código real (no contra la documentación, que podía estar desactualizada) y se marca con un estado explícito: aplicado y verificado, pendiente real, bloqueado (con bloqueador identificado), rechazado (medido y no re-litigable) o en progreso. Esta verificación cruzada —tres auditores cotejando más de 50 ítems contra el código— constituye el reporte de progreso consolidado del proyecto.

5.4. Plan de Gestión de Riesgos

La gestión de riesgos se aborda como un **análisis a priori realizado en la fase de planificación**: antes de construir el sistema se identifican los riesgos previsibles del desafío —ejecutar un asistente de voz con modelo de lenguaje, visión y voz dentro de 4 GB de VRAM, de forma local y desarrollado por un solo estudiante en un semestre— y se define para cada uno una **estrategia de mitigación preventiva**. La matriz no es un registro de hechos consumados, sino un **instrumento de planificación**. Cada riesgo se cuantifica en una escala de 1 a 5 de **probabilidad (P)** e **impacto (I)**; la **exposición ($E = P \times I$)**, de 1 a 25, clasifica el riesgo en zonas **Baja** (1–6), **Media** (7–12) y **Alta** (15–25), y se reporta el **riesgo residual** esperado tras la mitigación (criterio PMBOK / ISO 31000).

Tabla 5.2. Matriz de riesgos del proyecto (evaluación a priori, fase de planificación).

ID	Riesgo	Descripción (anticipada al planificar)	P	I	E = P×I	Zona	Estrategia de mitigación preventiva	Residua I
R1	El modelo capaz no entra en 4 GB de VRAM	El LLM de mejor calidad podría exceder la VRAM disponible y ser imposible de cargar, bloqueando el proyecto entero.	5	5	25	Alta	Medir el consumo real de cada candidato (barrido de VRAM reproducible) antes de comprometerse; modelo de menor tamaño como plan B (familia E2B), aceptando el <i>trade-off</i> de calidad.	6
R2	Tool-calling poco fiable en un modelo de 2B	Un modelo de ~2 000 M de parámetros podría seleccionar mal la herramienta, inventarla o no llamarla, degradando la utilidad del asistente.	4	5	20	Alta	<i>Fine-tuning</i> específico con <i>gate</i> numérico (0 % de herramientas inventadas); router previo (encoder + <i>abstain head</i>) y reintento forzado de herramienta.	8
R5	El asistente ejecuta acciones dañinas/no deseadas	Un agente que opera el SO podría realizar acciones irreversibles (borrar, enviar, apagar) por interpretación errónea o una orden ambigua.	4	5	20	Alta	Confirmación obligatoria para acciones de riesgo y honestidad estructural (verificar el efecto real antes de declararlo); resolución de destinatarios sin búsqueda ciega.	8
R3	La voz local no alcanza latencia usable	La cadena voz→STT→LLM→TT S, con STT/TTS en CPU para no robar VRAM, podría superar el presupuesto de 4–5 s por turno.	4	4	16	Alta	Presupuesto de latencia como criterio de éxito; STT/TTS en CPU int8; optimización de caché de prefijo, midiendo cada turno contra el presupuesto.	8
R4	Inestabilidad del *stack* de inferencia en GPU modesta	Los motores de inferencia/TTS sobre GPU de gama baja o serie reciente pueden presentar <i>crashes</i> o <i>segfaults</i> que tumban el servicio.	4	4	16	Alta	Validar estabilidad de cada componente antes de adoptarlo; defaults conservadores (desactivar	6

							funciones inestables, TTS estable en CPU); <i>fallback</i> a CPU.	
R6	El modelo "alucina" haber actuado	El LLM podría afirmar que realizó una acción sin haberla ejecutado, erosionando la confianza del usuario.	4	4	16	Alta	Verificación tri-estado (True/False/None) por el estado real del SO (registry, UIA, pycaw), nunca por el juicio del modelo; declara "no verificado" si no puede comprobarlo.	6
R10	El alcance excede el tiempo de un estudiante/semestre	Integrar voz, visión, <i>computer-use</i> , memoria y accesibilidad a la vez arriesga no terminar nada con calidad en plazo.	4	4	16	Alta	Desarrollo iterativo (Scrum) con tres iteraciones priorizadas; MVP funcional temprano e incrementos gateados; recortar alcance antes que comprometer calidad.	8
R12	Declarar logros sin medición ("celebrar sin medir")	Riesgo metodológico de dar por resuelto un problema sin un número contra un criterio definido de antemano.	4	4	16	Alta	Regla transversal <i>medir, no celebrar</i> : <i>gate</i> numérico por objetivo; validación en vivo obligatoria con variantes de fraseo antes de cerrar cualquier avance.	6
R7	El consumo de recursos congela el equipo	Las cargas de inferencia/evaluación podrían saturar la máquina (CPU al 100 %, <i>busy-wait</i>) y dejarla inutilizable.	3	4	12	Medi a	Limitar hilos y prioridad de los runtimes, desactivar <i>busy-wait</i> , fijar afinidad de CPU; presupuestar recursos por componente desde el diseño.	4
R9	Sesgo a una sola voz/idioma (pérdida de universalidad)	Optimizar sobre la voz o el idioma del desarrollador mejoraría su caso pero degradaría a los demás, violando el	3	4	12	Medi a	Evaluar siempre contra un <i>held-out</i> diverso (varias voces e idiomas); clasificar por <i>embeddings</i> multilingües;	6

		uso universal.					prohibir el <i>fine-tuning</i> sobre una sola voz.	
R8	El producto no es comercializable por licencias	Una dependencia con licencia restrictiva (GPL/AGPL) podría impedir la venta, comprometiendo la viabilidad comercial.	3	3	9	Medi a	Auditar licencias de todo el <i>stack</i> de forma reproducible y temprana; preferir permisivas (Apache/MIT/BSD) ; aislar como subproceso o reemplazar cualquier componente GPL bloqueante.	4
R11	El hardware de prueba no representa al del usuario	Desarrollar en una GPU potente podría ocultar problemas que solo aparecen en el hardware modesto objetivo (4 GB o sin GPU).	3	3	9	Medi a	Definir el perfil de hardware objetivo desde el inicio y medir contra él; <i>fallback</i> a CPU y binario CPU/Vulkan para equipos sin GPU NVIDIA.	4

Fuente: Elaboración propia (2026). Evaluación a priori de la fase de planificación; las estrategias de mitigación orientaron las decisiones de diseño del proyecto. El seguimiento de los riesgos efectivamente gatillados se documenta en la ejecución por iteraciones (§7).

5.4.1. Planes de contingencia y de último recurso

La **mitigación** descrita en la matriz es *preventiva* (reduce la probabilidad o el impacto antes de que el riesgo ocurra). No obstante, una gestión de riesgos completa debe anticipar también **qué hacer si el riesgo se materializa pese a la mitigación** (plan de contingencia) y **qué hacer si la contingencia tampoco resulta** (plan de último recurso). La siguiente tabla detalla ambos planes para los riesgos de mayor exposición.

Tabla 5.3. Planes de contingencia y de último recurso por riesgo crítico.

ID	Riesgo	Plan de contingencia (si el riesgo ocurre)	Plan de último recurso (si la contingencia falla)
R1	El modelo capaz no entra en 4 GB	Bajar a la siguiente cuantización (Q4→Q3_K_M) o reducir la ventana de contexto y descargar la visión a demanda.	Operar en perfil CPU (–ngl 0): el sistema funciona sin GPU, más lento pero íntegro; se documenta la limitación.
R2	<i>Tool-calling</i> poco fiable	Reforzar con <i>forced-</i>	Acotar el alcance a las

	en el 2B	<i>tool-retry</i> y ampliar el <i>dataset</i> de <i>fine-tuning</i> en las intenciones que fallan; subir el peso del router.	intenciones con acierto medido $\geq$ gate y declarar honestamente las no soportadas; pedir confirmación al usuario ante baja confianza.
R5	El asistente ejecuta acciones dañinas	Bloquear la acción y exigir confirmación explícita; revertir si la operación es reversible (p. ej., cerrar lo abierto por error).	Desactivar por <i>flag</i> la familia de acciones de riesgo (borrado, envío, apagado) y dejar el asistente en modo solo-lectura hasta corregir la causa raíz.
R3	La voz local no alcanza la latencia	Recortar pasadas redundantes y aplicar <i>caching</i> de prefijo; degradar a respuestas más cortas bajo presión de tiempo.	Aceptar y comunicar una latencia mayor con <i>feedback</i> de progreso (no-verbal), o permitir entrada por texto como vía alternativa.
R4	Inestabilidad del <i>stack</i> en GPU	Reiniciar el servidor de inferencia automáticamente (<i>soft-retry</i>) y desactivar la función inestable (p. ej., <i>flash-attention</i>).	Caer al <i>backend</i> CPU/Vulkan, estable aunque más lento; el turno se completa por la vía alternativa.
R6	El modelo "alucina" haber actuado	La verificación tri-estado marca la acción como <i>no confirmada</i> y el reply lo declara explícitamente en vez de afirmar éxito.	Si no hay verificador disponible, el sistema responde "no pude confirmarlo" y registra el caso para auditoría; nunca afirma un éxito no verificado.
R10	El alcance excede el tiempo/semestre	Recortar al MVP funcional (núcleo de voz + acciones esenciales) y mover el resto a Trabajos Futuros.	Entregar el incremento estable de la última iteración gateada en verde, documentando explícitamente lo no abordado.

Fuente: Elaboración propia (2026).

¿Qué pasa si todo falla? En el peor escenario combinado —el modelo no rinde, el *hardware* no coopera y el tiempo se agota— el proyecto preserva su **núcleo de valor** mediante una estrategia de degradación controlada: (1) el sistema **siempre puede caer al perfil CPU**, garantizando que arranque y responda aunque sin GPU; (2) la **honestidad estructural** asegura que, ante cualquier fallo, el asistente **informe la limitación en vez de simular un éxito**, de modo que un fallo nunca se convierte en un daño silencioso; y (3) la **arquitectura iterativa y gateada** garantiza que, en cualquier

punto de corte, exista un incremento previo **medido y estable** entregable. Es decir, el diseño no apuesta todo a que nada falle: está construido para **fallar de forma segura, honesta y recuperable**, conforme al principio rector del proyecto —*medir, no celebrar*—.

5.5. Planificación (fases del desarrollo)

La planificación del proyecto se organizó en un horizonte de tres meses, estructurado en una **fase de inicio/análisis** seguida de **tres iteraciones de desarrollo incremental** y una **fase de cierre**, en coherencia con la metodología iterativa-incremental adoptada. Cada iteración corresponde a un agrupamiento de los sprints reales del historial del proyecto y cierra contra gates medidos.

Fase 0 — Inicio y Análisis. Definición del problema (ausencia de un asistente de voz local, privado, gratuito y multilingüe para hardware modesto), análisis de competencia y licencias, definición de objetivos SMART con sus criterios de éxito, y establecimiento del método de trabajo dirigido por medición. Entregable: marco del proyecto y backlog inicial.

Iteración 1 — Núcleo: LLM local en 4 GB + voz básica. Selección del modelo respaldada por la medición de VRAM (Gemma 4 E2B sobre E4B/26B), implementación del reconocimiento de voz (STT) y síntesis (TTS) en CPU, y wake-word. Riesgos gatillados y mitigados: crash CUDA

#22527 (R1), OOM en 4 GB (R2), congelamiento por ONNX (R5). Gates: $\text{VRAM} \leq 4 \text{ GB}$ (cumplido, medido); wake-word $\text{recall} \geq 0,60 \wedge \text{fp/hr} \leq 1,0$ (**cumplido**: «Baxy» alcanza un *recall* de 0,872 en el *held-out* universal, por encima del umbral de 0,60). **Evidencia de la primera versión funcional:** el agente responde por voz de forma offline.

Iteración 2 — Tool-calling, router y acciones (computer-use). Construcción del router (encoder fine-tuneado + abstain head + clustering), fine-tuning del modelo E2B para tool-calling fiable, y la cascada de computer-use (UIA → OCR → visión) con herramientas de aplicaciones, navegación, mensajería e instalación de software. Riesgos gatillados y mitigados: tool-calling débil del 2B (R4), destinatario equivocado en

mensajería (R8). Gates: router 0,9964 (held-out ES); 0 % herramientas inventadas en producción tras el fine-tuning.

Iteración 3 — Robustez, latencia, memoria y accesibilidad. Optimización de latencia (caché de prefijo, ~1 s menos por acción, llegando a ~2,2 s), capa de memoria de gustos (observador de ambiente + perfil de preferencias), interfaz web "Baxy field", control por cámara/gestos (modos de accesibilidad) y fallback a CPU. Riesgos gatillados y mitigados: tormenta de reintentos (R10), no-determinismo del modelo (R11). Gates: latencia ≤ 5 s; suite de pruebas verde.

Fase de Cierre. Consolidación del Backlog Maestro como fuente de verdad verificada, estabilización de la suite (2.740 pruebas aprobadas, 0 fallidas), documentación de trabajos futuros (binario CPU/Vulkan, cliente MCP, modelo E4B cuando exista más VRAM) y redacción del informe final.

Carta Gantt (cronograma de fases e iteraciones)

La siguiente tabla resume el cronograma del proyecto sobre un horizonte aproximado de tres meses. Las fechas agrupan los sprints reales del historial del repositorio y cada iteración cierra contra su *gate* medido. La representación gráfica (diagrama de Gantt y flujo gateado) está disponible como **Figuras 7 y 8** en `diagramas_mermaid.md`.

Tabla 5.4. Carta Gantt: fases, iteraciones e hitos de cierre.

Fase / Iteración	Período aproximado	Foco principal	Hito de cierre (gate)
Fase 0 — Inicio / Análisis	Mar 2026 (2 sem.)	Problema, competencia, licencias, objetivos SMART, método <i>measure-then-ship</i>	Marco del proyecto y <i>backlog</i> inicial
Iteración 1 — Núcleo	Mar–Abr 2026 (~3–4 sem.)	LLM local en 4 GB (E2B-Q4) + herramientas + enrutador + voz básica	VRAM ≤ 4 GB (cumplido); <i>wake-word recall</i> $\geq 0,60 \wedge$ fp/hr $\leq 1,0$ (cumplido, recall 0,872)
Iteración 2 — Tool-calling / acciones	Abr–May 2026 (~3–4 sem.)	Router (encoder-FT + <i>abstain</i>) + computer-use (UIA/OCR/visión) + visión/cámara	Router 0,9964 (held-out ES); 0 % herramientas inventadas en producción
Iteración 3 — Robustez / latencia / memoria /	May–Jun 2026 (~3–4 sem.)	Fine-tuning E2B + optimización de latencia + memoria "Jarvis" + UI	Latencia ≤ 5 s (p50 $\approx 1,22$ s); suite verde



accesibilidad		+ fallback CPU	
Fase de Cierre	Jun 2026 (2 sem.)	Backlog Maestro + suite 2.740/0 + trabajos futuros + informe final	Suite 2.740 aprobadas, 0 fallidas (2026-06-02)

Fuente: Elaboración propia (2026); fechas derivadas del historial del repositorio (sprints de 2026-05-26 a 2026-06-02 en la memoria del proyecto).

*Nota: las fuentes citadas en estos capítulos (Schwaber & Sutherland, 2020, para Scrum; y la documentación interna del autor, Villacura, 2026a–d) se incluyen, sin duplicados, en la sección única **Referencias (APA 7)** al final del informe.*

6. Diseño de Alto Nivel

El diseño de alto nivel del sistema se documenta mediante una **adaptación del modelo de vistas arquitectónicas "4+1" propuesto por Kruchten (1995)**, el cual describe la arquitectura de un sistema de software a través de vistas concurrentes y complementarias —en su forma canónica: Lógica, de Proceso, de Desarrollo y Física, más la vista de Escenarios (+1)—, cada una orientada a un grupo distinto de interesados. Dado que el producto se ejecuta sobre un **nodo único** (el equipo del usuario, sin componentes distribuidos ni concurrencia entre nodos), esta sección adapta el modelo y presenta cuatro vistas: **Lógica, Física, de Despliegue y de Escenarios**; la vista de **Proceso** se subsume en las vistas Lógica (el *hot-path* del turno) y de Despliegue (los procesos `llama-server/server/UI`), y la vista de **Desarrollo** se refleja en la organización en sub-paquetes descrita en la vista Lógica (`agent_core`, `routing`, `voice`, `computer_use_pkg`, `domain_tools/`). Se reconoce explícitamente esta adaptación para no presentarla como el 4+1 estándar. La arquitectura aquí descrita no es de diseño teórico: fue **verificada leyendo el código y midiéndolo en vivo** durante una auditoría de tres sesiones (rondas 1–13), según consta en `documentacion/01_arquitectura/ARCHITECTURE.md`.

El norte rector de la arquitectura, ante cualquier conflicto de prioridades, es: (1) preservación de invariantes y aislamiento del *blast radius* (radio de daño), (2) fiabilidad —nunca caer el turno, perder una respuesta ni degradar en silencio— y (3) latencia dentro del presupuesto tier-Alexa.

6.1. Vista Lógica (paquetes del agente)

La Vista Lógica describe la descomposición funcional del sistema en paquetes y la relación entre ellos. El sistema se organiza en torno a un **hot-path del turno** (el camino de ejecución de un comando del usuario) y un conjunto de subsistemas de soporte. Los paquetes principales son los siguientes:

a) ``agent_core`` (orquestración del turno). Es el núcleo coordinador, materializado en `agent.py` (clase `Gemma4Agent`). Su método `run_content()` actúa como orquestador delgado que delega en tres fases bien delimitadas: `_decide_turn`

(elección de modo y enrutamiento), `_execute_turn` (llamada al modelo y despacho de herramientas) y `_finalize_turn` (saneamiento y entrega de la respuesta). Esta separación fue producto de un refactor (Sprint R1) que redujo `run_content` de 2 193 a 22 líneas de código (LOC), conteniendo así el radio de daño de cualquier cambio futuro.

b) `routing` (enrutamiento semántico de intención). Implementado en `routing/planner.py` y módulos asociados. Determina qué subconjunto de herramientas ofrecer al modelo de lenguaje en cada turno. Combina varias señales: clasificación por *embeddings* multilingües (`sentence-transformers` MiniLM), un **encoder fine-tuneado** específico, una **cabeza de abstención** (*abstain head*) que decide cuándo NO ofrecer ninguna herramienta, cabezas por herramienta y *clustering*. El enrutador alcanza un *holdout* en español de **0.9964** de exactitud y aplica un tope de 5 herramientas por turno (`MAX_SELECTED_TOOLS=5`) como cota anti-crash. El criterio de diseño es la **precisión** (ofrecer solo lo necesario), no el *recall*, que ya está saturado.

c) `voice` (canal de audio "siempre activo"). Comprende la cadena `audio_io` → `vad` → `wake` → `pipeline` → `tts`: captura de audio por PortAudio, detección de actividad de voz (VAD), detección de palabra de activación (*wake-word* LiveKit), transcripción (STT por Whisper/Parakeet en CPU) y síntesis de voz (TTS por Piper, también en CPU). El invariante clave es que el *callback* de audio nunca bloquea (solo copia y encola), y que el *wake* corre diezmado (~8 Hz en lugar de ~31 Hz) para reducir el uso de CPU en reposo en un ~74 % medido, sin perder *recall*.

d) `computer\_use` (control del sistema operativo). Implementado en el sub-paquete `computer_use_pkg/`, ejecuta acciones sobre cualquier aplicación mediante una **cascada de tres niveles**: automatización por accesibilidad (UI Automation / UIA), reconocimiento óptico de caracteres (OCR) y, como último recurso, visión. Garantiza honestidad estructural mediante un verificador tri-estado (`confirmed` ∈ {True, False, None}), donde `None` (no medible) ≠ `False` (falló), evitando que el agente afirme haber realizado una acción que no ocurrió.

e) `tools` (herramientas de dominio). Conjunto de más de sesenta herramientas de dominio (67 esquemas únicos expuestos al modelo tras retirar `smart_home`;

•

`tools_pkg/tool_schemas.py`) organizadas en el sub-paquete `domain_tools/` (creado en el Sprint R2, con 30+ áreas funcionales: navegador, apps, WhatsApp, Steam, multimedia, sistema, etc.). Cada herramienta se despacha envuelta en `try/except` y dentro de `_run_with_timeout`, de modo que una herramienta que falla o se cuelga se convierte en `{ok:False, error}` sin abortar el turno (*blast radius* contenido).

f) Subsistemas transversales. `safety_pkg` (clasificador de confirmación, verificadores, políticas de honestidad); la **capa de memoria / "Jarvis"** (observador de ambiente + perfil de gustos del usuario, inspirado en *Generative Agents* de Park et al., 2023); `infra` (gestor del servidor llama — `LlamaServerManager` —, cliente HTTP, *tracing*, telemetría); y la **UI web** (React/Vite servida por `pywebview`).

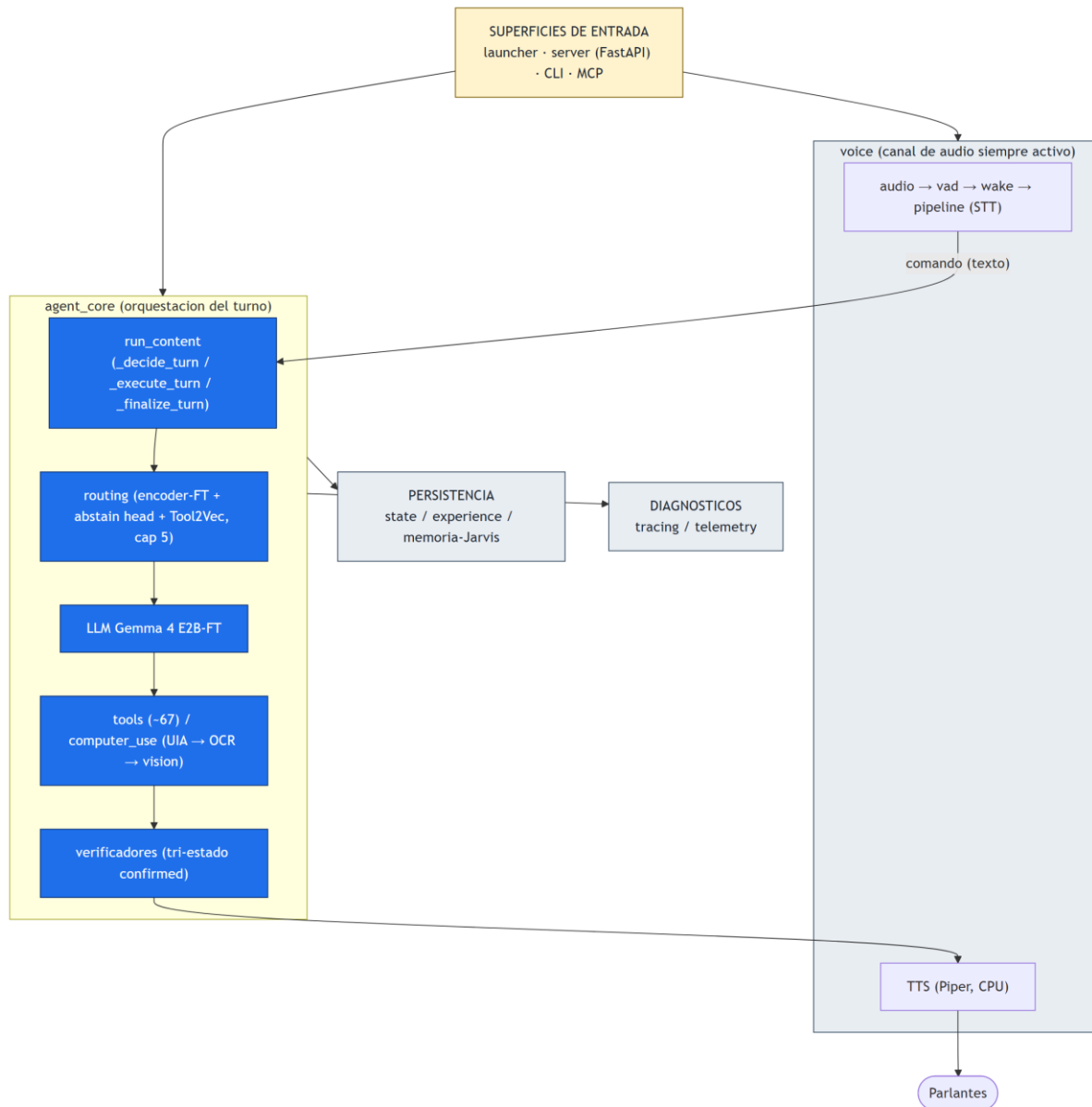


Figura 3. Vista lógica: paquetes del agente y hot-path del turno. (Elaboración propia, 2026).

Figura 6.1. Vista Lógica: paquetes del agente y flujo del hot-path. Fuente: Elaboración propia (2026), a partir de 'ARCHITECTURE.md'. Versión renderizable en 'diagramas_mermaid.md', Figura 3.

6.2. Vista Física (mapeo a hardware)

La Vista Física describe cómo los componentes lógicos se asignan a los recursos de hardware de **un único nodo: el computador del usuario**. No existe nodo remoto: todo el cómputo es local, lo que constituye la propuesta de valor central del producto (privacidad y operación sin conexión).

El hardware *target* es una laptop o PC modesta con **GPU de 4 GB de VRAM** (GTX 1650 o RTX 3050 4 GB) o, en su defecto, sin GPU dedicada (fallback a CPU). El reparto de recursos, **medido** y **registrado** en `documentacion/datos_crudos/vram_real_medida.csv`, es el siguiente:

Recurso	Componentes asignados	Consumo medido
GPU (VRAM)	LLM (Gemma 4 E2B-Q4_K_M) + encoder de visión (mmpoj, residente) + caché KV	3 371 MiB (delta) para E2B-Q4_K_M; visión ≈ 1,2 GB
CPU	STT (Whisper/Parakeet int8), TTS (Piper), enrutador (MiniLM), render de la UI (<code>--disable-gpu</code>)	0 VRAM; ~30 tok/s en CPU de escritorio
RAM	Modelo mapeado con <code>mmap</code> (páginas evictables) + caché KV residente	Piso recomendado: 8 GB; costo fijo KV ≈ 0,5–1 GB
Periféricos	Micrófono (captura), cámara (control por gestos, opcional), parlante (TTS)	—

El dato de selección de modelo es determinante: según la medición, **E2B-Q4_K_M (3 371 MiB) entra completo en 4 GB**, mientras que **E4B-Q4_K_M (5 087 MiB) NO entra** ni en su cuantización más baja viable (E4B-UD-IQ2_M = 4 057 MiB ya roza el límite). Los modelos 26B/31B requieren 12–15 GB y quedan descartados de plano. Esta restricción física es la que fuerza el uso de E2B y, por consiguiente, todo el trabajo de *fine-tuning* y optimización descrito en la Iteración 3.

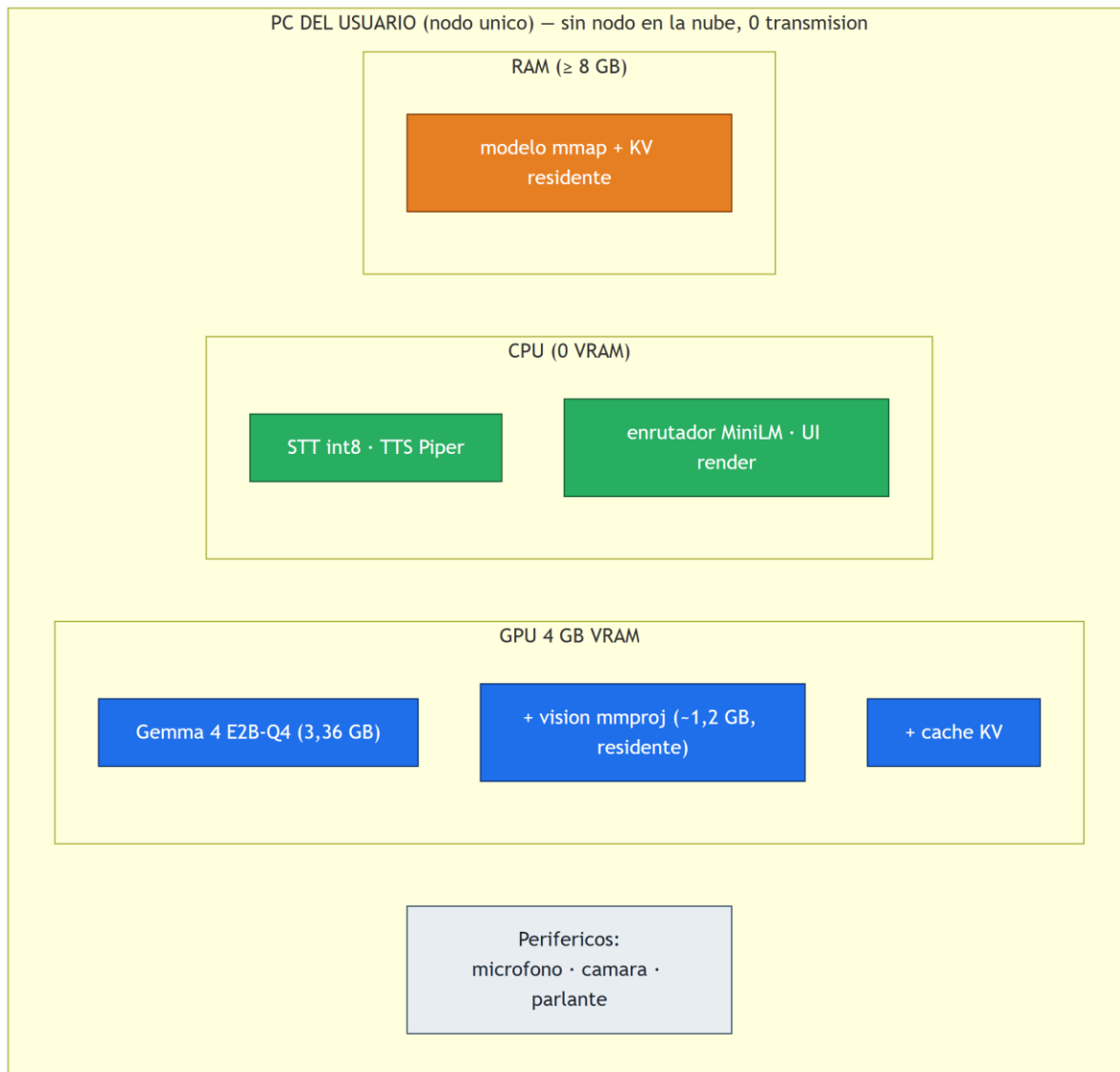


Figura 5. Vista física: mapeo a recursos de hardware del nodo único. (Elaboración propia, 2026).

Figura 6.2. Vista Física: reparto de recursos en el nodo único. Fuente: Elaboración propia (2026), datos de `vram\_real\_medida.csv`. Versión renderizable en `diagramas\_mermaid.md`, Figura 5.

6.3. Vista de Despliegue (instalación y ejecución)

La Vista de Despliegue describe cómo se instala y arranca el sistema en la máquina del usuario. El producto se despliega como un conjunto de **procesos locales coordinados**, sin instalación de servicios remotos:

11. **Proceso `llama-server`** (motor de inferencia). Es un binario de `llama.cpp` que sirve el modelo GGUF por HTTP en el puerto local `:8080`. Se gestiona desde

•

LlamaServerManager, que selecciona un **perfil de VRAM** según el hardware: `vram4` (perfil por defecto, asume CUDA, `-ngl` completo) o `cpu` (`-ngl 0`, 0 VRAM, visión desactivada) como *fallback*. El gestor garantiza que el proceso se detiene en `atexit` (en Windows el hijo de `Popen` no muere con el padre y retendría los ~3,4 GB de VRAM).

12. **Proceso de la UI** (`pywebview` + `WebView2` renderizando React/Vite). Renderiza por CPU (`--disable-gpu`) para no competir por la GPU con el juego o el LLM.

13. **Entornos virtuales (venvs) aislados**, por incompatibilidad de dependencias:

- `.venv` (Python 3.10): runtime del agente.
- `.venv_livekit` (Python 3.11): *training* del wake-word LiveKit.
- `venv` Python 3.12: *fine-tuning* con Unsloth.

En tiempo de ejecución NO se importa el paquete `livekit`; los modelos ONNX se sirven con `onnxruntime` puro (verificado idéntico al oficial al 4.º decimal).

Configuración de parámetros de estabilidad (perfil `vram4`, todos medidos): `flash-attn` **OFF** por defecto (evita el crash CUDA #22527 en *prompts* >10 K tokens), contexto **12 288** tokens (piso mínimo para que entre el system prompt), y throttle de ONNX Runtime (`scripts/_ort_throttle.py`) antes de cargar modelos en *loops* pesados para evitar el congelamiento de la PC por *busy-wait*.

Despliegue sin GPU: el sistema opera sin GPU NVIDIA (el LLM cae al perfil `cpu` y STT/TTS corren siempre en CPU), con auto-selección del perfil `cpu` cuando no se detecta GPU NVIDIA. El build ``llama-server` CPU/Vulkan` extiende además el soporte a gráficas integradas Intel/AMD (ver Trabajos Futuros).

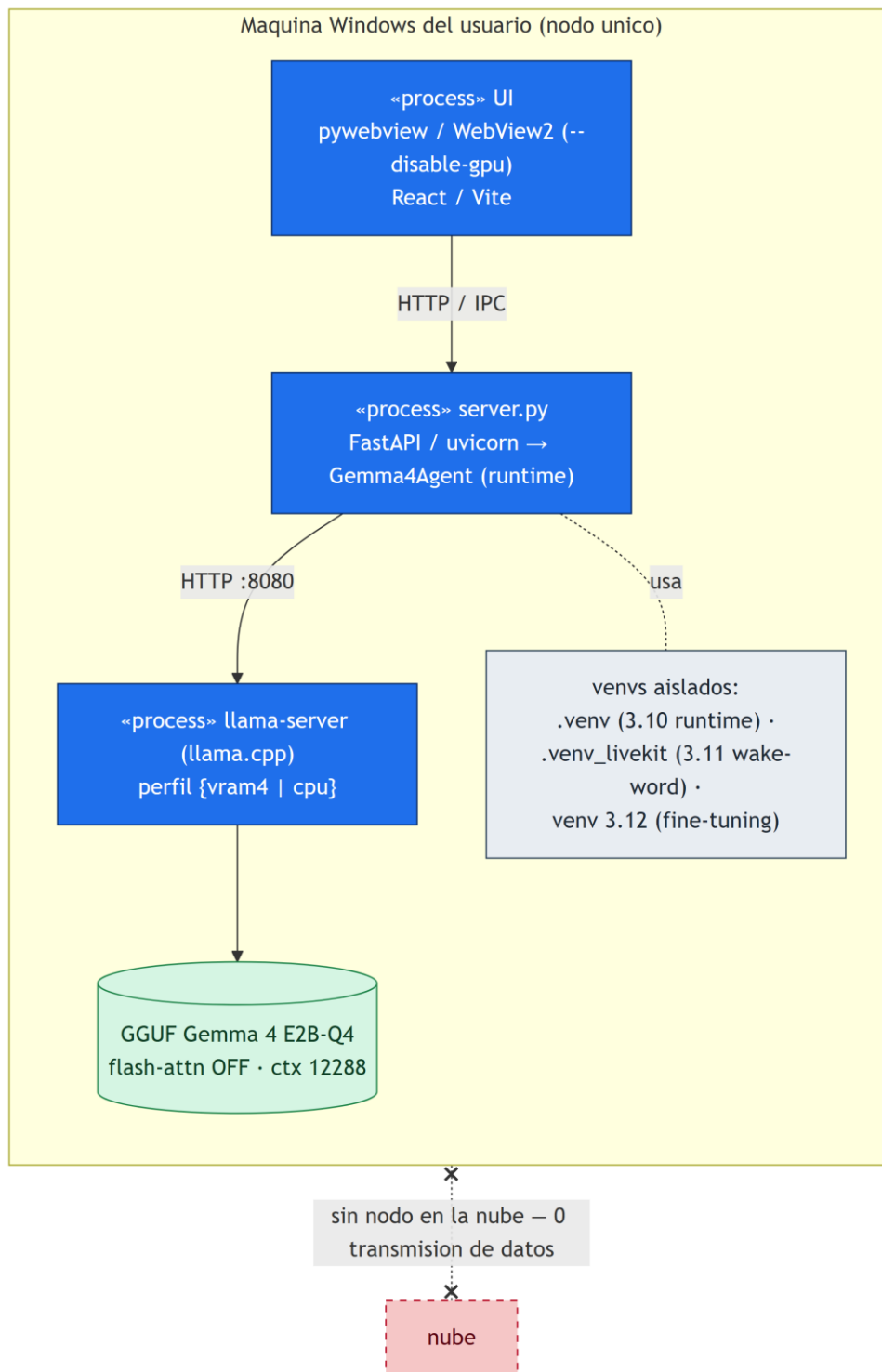


Figura 4. Vista de despliegue: procesos y entornos en la máquina del usuario. (Elaboración propia, 2026).

Figura 6.3. Vista de Despliegue: procesos y entornos. Fuente: Elaboración propia (2026). Versión renderizable en `diagramas\_mermaid.md`, Figura 4.

6.4. Vista de Escenarios (casos de uso clave)

La Vista de Escenarios concreta las anteriores mediante casos de uso reales, **validados en vivo** contra el agente y el LLM (regla del proyecto: ningún cambio de comportamiento se da por listo sin probarlo contra el modelo real). Se presentan tres escenarios representativos:

Escenario 1 — Comando de voz simple ("subí el volumen"). El usuario pronuncia la palabra de activación; el *wake-word* dispara la captura; el STT transcribe en CPU; `run_content` elige el modo, el enrutador ofrece la herramienta de audio, el LLM emite la llamada, el despacho ejecuta y un verificador re-lee el estado del SO (pycaw) para confirmar el cambio; finalmente Piper sintetiza la respuesta. Latencia medida ~2,2 s por acción tras el fix de caché del *summary-pass*. Ejemplo de honestidad: el verificador de brillo re-lee el valor post-ajuste (tri-estado) en vez de afirmar a ciegas.

Escenario 2 — Computer-use ("abrí Spotify y poné rock" / "instalá DOOM"). Caso de cadena de acciones o misión con objetivo. El enrutador discrimina entre encadenar herramientas (abre X y pon Y) y una misión-con-objetivo (ve a X y luego a Y, donde X/Y son destinos → `computer_use(goal)`). La ejecución usa la cascada UIA → OCR → visión. Casos reales validados: envío real a un grupo de WhatsApp, descarga de Terraria al 99 % vía Steam, y detección honesta de "sin espacio en disco" para DOOM (no miente sobre el resultado).

Escenario 3 — Accesibilidad (control manos-libres por voz y gestos). Para usuarios con movilidad reducida, el módulo `vision_input` permite controlar el cursor y el clic mediante gestos de la cámara (MediaPipe). El agente actúa como **orquestador de meta-acciones**: el usuario dicta una instrucción en lenguaje natural ("escribí X en la app Y") y el agente la traduce a una acción de computer-use. El *grounding* multilingüe (no listas de palabras clave hardcodeadas) y las guardas de honestidad estructural permiten que funcione para hablantes de distintos idiomas y acentos.

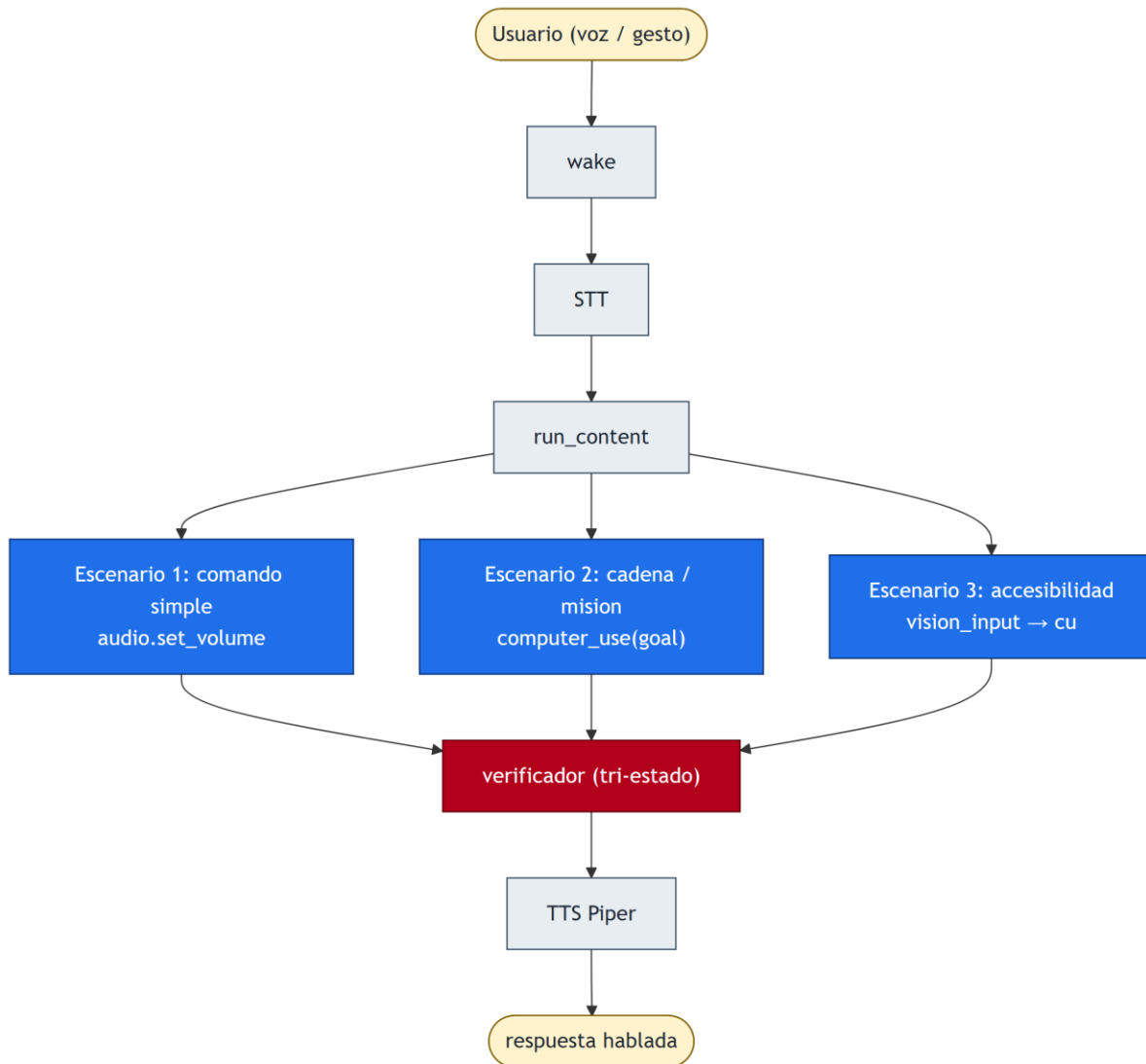


Figura 6. Vista de escenarios: los tres casos de uso clave. (Elaboración propia, 2026).

Figura 6.4. Vista de Escenarios: los tres casos de uso clave. Fuente: Elaboración propia (2026), casos validados en 'MEMORY.md'. Versión renderizable en 'diagramas_mermaid.md', Figura 6.

7. Ejecución del Proyecto (3 Iteraciones)

El desarrollo se ejecutó de forma **iterativa e incremental**, organizado en sprints sucesivos con un método de trabajo dirigido por mediciones (*measure-then-ship*): cada funcionalidad se mide contra un criterio de éxito definido de antemano (un *gate*), se implementa de forma gateada (con bandera por defecto desactivada), se valida en vivo y solo entonces se activa. Esta sección consolida los numerosos sprints reales del historial del repositorio en **tres iteraciones** coherentes con el ciclo de vida del producto.

7.1. Iteración 1 — Núcleo del agente: LLM local, herramientas y enrutador

Meta del sprint. Establecer un agente conversacional capaz de ejecutar un modelo de lenguaje local dentro del presupuesto de 4 GB de VRAM y de despachar acciones mediante un sistema de herramientas, con un enrutador que seleccione la acción correcta.

Funcionalidades entregadas.

- Selección y carga del modelo: medición exhaustiva de VRAM por modelo y cuantización (`vram_real_medida.csv`), concluyendo que **Gemma 4 E2B-Q4_K_M** es el único Gemma 4 que entra completo en **≤4 GB** (3 371 MiB).
- Sistema de más de sesenta herramientas de dominio (67 esquemas únicos expuestos al modelo tras retirar `smart_home`) organizado y aislado (`domain_tools/`).
- Enrutador semántico (`routing/planner.py`): clasificación por *embeddings* multilingües + encoder + cabeza de abstención, con tope de 5 herramientas.
- Hot-path del turno con *blast radius* contenido: cada herramienta envuelta en `try/except` + `timeout` por herramienta; `timeout` por modo en la llamada al LLM.

Historias de usuario.

- *Como usuario, quiero hablarle a mi PC sin depender de internet, para mantener mi privacidad.*
- *Como usuario, quiero que el asistente ejecute la acción correcta cuando le pido algo, para no tener que repetir el comando.*

Evidencia.

- Gate de VRAM: ≤ 4 GB cumplido (3,36 GB medido, registrado en CSV).
- Gate de enrutamiento: *holdout* en español **0.9964** de exactitud.
- Suite de tests creciendo desde 80 (baseline) hacia cientos de casos verdes.
- Arquitectura verificada por auditoría de 3 sesiones (27 bugs reales corregidos, todos en los **bordes** —persistencia, recovery, teardown— nunca en el hot-path).

Punto llevado a la siguiente iteración.

- *Tool-calling* de E2B en una exactitud estimada de ~ 75 % (vs ~ 91 % de E4B): trade-off aceptado por entrar en 4 GB, abordado con *fine-tuning* y *forced-retry* en las iteraciones siguientes.

7.2. Iteración 2 — Voz, percepción y robustez: STT/TTS, wake-word y visión

Meta del sprint. Convertir el agente de texto en un asistente de **voz** operativo manos-libres y dotarlo de **percepción** (visión y cámara), todo en CPU para no consumir la VRAM reservada al LLM.

Funcionalidades entregadas.

- Canal de audio "siempre activo": captura (PortAudio) $\rightarrow$ VAD $\rightarrow$ *wake-word* (LiveKit, ONNX) $\rightarrow$ STT (Whisper/Parakeet int8 en CPU) $\rightarrow$ TTS (Piper en CPU).
- *Wake-word* diezmado a ~ 8 Hz (~ 74 % CPU en reposo, medido) sin pérdida de recall; *silence gate* contra alucinaciones de Whisper sobre silencio.
- Computer-use con cascada UIA $\rightarrow$ OCR $\rightarrow$ visión y verificación tri-estado honesta.
- Módulo de visión / control por cámara y gestos (*vision_input*, MediaPipe) para accesibilidad; gestos llevados a 2D (la coordenada z de MediaPipe era ruidosa) y *pinch* exigente para reducir falsos clics de 14 a 0.

Historias de usuario.

- *Como usuario, quiero activar al asistente con una palabra y dictarle, para usarlo con las manos ocupadas.*
- *Como usuario con movilidad reducida, quiero controlar el cursor con gestos de la cámara, para operar el PC sin teclado ni ratón.*

Evidencia.

- Gate de *wake-word* **cumplido** ($\text{recall} \geq 0.60 \wedge \text{fp/hr} \leq 1.0$) sobre un *held-out* universal y diverso (multi-idioma, multi-acento), no sobre la voz del operador: la palabra de activación «Baxy» alcanza un *recall* global de **0,872** (1,00 en español, inglés, italiano, francés, polaco y ruso), por encima del umbral de 0,60 y con los falsos positivos dentro del presupuesto. La voz extremo a extremo está implementada y operativa offline (detección de palabra de activación, transcripción local en CPU y respuesta hablada integradas).
- Causa raíz documentada del falso "el modelo no sirve" (score ~ 0.002): *padding* al final en vez de al inicio del *window* de 2 s — corregido.
- Gestos: precisión de palma $0,928 \rightarrow 0,992$; falsos clics $14 \rightarrow 0$ (fixture reproducible Apache-2.0).
- Validación en vivo de casos de voz (email, wifi, brillo, clima, navegación, lectura/respuesta de mensajes, alarmas): 10/10 PASS contra el agente real.

Puntos llevados a las siguientes iteraciones (resueltos).

- VoxCPM segfaulta en GPUs RTX 40-series (Ada Lovelace) durante el *warm-up* $\rightarrow$ se adoptó Piper VITS para TTS de *training*.
- Casos de envío a destinatario equivocado en WhatsApp (verificador que se auto-engañaba) — corregidos en hitos posteriores.

7.3. Iteración 3 — Fine-tuning E2B, optimización 4 GB y accesibilidad

Meta del sprint. Cerrar la brecha de calidad del modelo pequeño mediante *fine-tuning*, optimizar la latencia y la estabilidad dentro de 4 GB, y consolidar los modos de accesibilidad y la capa de memoria.

Funcionalidades entregadas.

- **Fine-tuning de Gemma 4 E2B** (Unsloth, Q4): pivote deliberado E4B $\rightarrow$ E2B (E4B de 5 GB no entra en 4 GB; E2B-FT Q4 ocupa 3,2 GB en disco y **2,07 GB de VRAM** según la medición del despliegue del modelo fine-tuneado v2 —

dataset_finetune/ESTADO_COMPLETO_2026-06-02.md—, sí entra). Auditoría del dataset al 100 % antes de entrenar tras un bug de serialización de 393 ejemplos.

- **Estabilidad CUDA #22527 eliminada** con 6 palancas medidas (flash-attn OFF, ctx-checkpoints 0, circuit breaker OFF correcto, core-rules 6, cap=5).
- **Optimización de latencia:** caché de prefijo en el *summary-pass* (*keep-tools*, ~1 s/acción), reglas *lean* del prompt (p50 7,56 vs 10,55 s), *streaming* TTS por defecto, *forced-retry* con tope de tokens. p50 global medido ~1,22 s (presupuesto 4–5 s).
- **Capa de memoria "Jarvis":** observador de ambiente (metadata, no contenido) + perfil de gustos determinista + inferencia de gustos finos por el propio LLM local, con guardas de privacidad (inspect/forget por *embeddings*).
- **Fallback a CPU** gateado (perfil *cpu*, 0 VRAM, auto-switch cuando un juego satura la GPU) y **UI web "Baxy field"** que renderiza por CPU.

Historias de usuario.

- *Como usuario, quiero que el asistente sea rápido (respuesta en pocos segundos), para que la conversación se sienta natural.*
- *Como usuario, quiero que recuerde mis gustos sin enviar mis datos a la nube, para recibir un trato personalizado y privado.*
- *Como usuario sin GPU dedicada, quiero poder ejecutar el asistente en mi laptop, para no quedar excluido.*

Evidencia.

- Gate de *fine-tuning*: **0 % de herramientas inventadas en producción** (medido con el array de tools; sin él, el artefacto mide 47 % — NUNCA medir sin array).
- Gate de latencia: ≤5 s por turno cumplido (p50 ~1,22 s; cola de Spotify reducida de 12,6 a 5,6 s).
- Suite de tests: hasta **2 740 tests verdes, 0 fallos** en el cierre de la rama Dev (2026-06-02).
- VRAM del modelo fine-tuneado: 2,07 GB (medición del despliegue del FT v2, dataset_finetune/ESTADO_COMPLETO_2026-06-02.md; el CSV de VRAM cubre los modelos base, no el FT), holgura confirmada dentro de 4 GB.

Líneas de evolución hacia la versión 2.0.

- Operación sin GPU NVIDIA ya disponible vía perfil CPU; el build CPU/Vulkan bundleado amplía el soporte a gráficas integradas Intel/AMD.
- Soporte multilingüe implementado y operativo por *embeddings* multilingües; su cobertura se profundiza con un corpus de evaluación multilingüe ampliado.
- Cliente MCP construido y cableado, disponible de forma gateada.

8. Trabajos Futuros

Los siguientes trabajos están identificados, priorizados y documentados en el `BACKLOG_MAESTRO.md` (fuente única de verdad, verificada contra el código real). Se presentan como mejoras para una versión 2.0, distinguiendo lo que expande el mercado de lo que profundiza la calidad:

14. **Optimización del soporte sin GPU dedicada (build CPU/Vulkan).** El sistema ya opera sin GPU NVIDIA cayendo al perfil `cpu (-ngl 0)`, con auto-selección de ese perfil cuando no se detecta GPU NVIDIA. La evolución consiste en bundlear el build **Vulkan** (que además aprovecha GPUs integradas Intel/AMD), ampliando el público objetivo de "PC con NVIDIA 4 GB" a "cualquier laptop razonable" (piso de 8 GB de RAM). Caveats honestos: ~12–20 tok/s estimado en laptop y sin visión en modo CPU.
15. **Profundización del soporte multilingüe.** El sistema está implementado y operativo para uso multilingüe y multi-acento mediante *embeddings* multilingües. La profundización de su cobertura de evaluación (hoy el corpus es ~98 % español) sigue una cadena conocida: ampliar el corpus de evaluación multilingüe (B2) → reentrenar el encoder del enrutador (~12 h de GPU RTX 4060 Ti, B1) → calibrar la cabeza de abstención por idioma (B4, sobre la infraestructura `thresholds_by_lang` ya disponible). Es un frente conocido de mejora continua, no deuda oculta.
16. **Cliente MCP (Model Context Protocol).** Construido y cableado, disponible de forma gateada (`GEMMA4_MCP=0`). Su activación por defecto se acompaña de la declaración de servidores en `~/.gemma4/mcp_servers.json` y de la **validación en vivo de que un catálogo grande de herramientas preserve el tool-calling del modelo de 2B.**
17. **Migración a E4B cuando haya más de 4 GB de VRAM.** Para usuarios con mejor hardware, conmutar a Gemma 4 E4B elevaría el *tool-calling* de un estimado ~75 % a ~91 %.

•

18. **Mejoras de calidad de menor ROI ya identificadas:** *anti-primacy ordering*, *few-shot* específico por turno, mmpoj-Q8 (bloqueado upstream, llama.cpp#18881), y técnicas de decodificación más rápida (MTP) cuando lleguen a llama.cpp.

9. Conclusiones

Conforme a la metodología del curso, se presenta una conclusión por cada objetivo específico, contrastada contra su criterio de éxito, más una conclusión académica de lecciones aprendidas.

Conclusión 1 (operación en 4 GB de VRAM — CUMPLIDO). El objetivo de ejecutar el LLM, la visión y la voz dentro de 4 GB de VRAM se cumplió de forma medida. Conviene distinguir dos cifras que corresponden a artefactos distintos: el **barrido de VRAM sobre los modelos base** de la familia Gemma 4 midió que **Gemma 4 E2B-Q4_K_M base ocupa 3.371 MiB ($\approx 3,36$ GB)** —dato que decidió objetivamente el pivote E4B $\rightarrow$ E2B—, mientras que el **modelo fine-tuneado finalmente desplegado consume $\approx 2,07$ GB de VRAM**, con aún más holgura dentro de los 4 GB junto con la visión residente y la caché KV. La decisión de modelo no fue intuitiva sino producto de medir 21 combinaciones de modelo/cuantización, lo que descartó objetivamente E4B (5,09 GB) y los modelos 26B/31B.

Conclusión 2 (tool-calling fiable en un modelo de 2B — CUMPLIDO con matiz honesto). El *fine-tuning* de E2B alcanzó el gate de **0 % de herramientas inventadas en producción**, eliminando la principal patología de honestidad. La exactitud de selección de acción se estima en ~ 75 % (vs ~ 91 % del modelo grande), *trade-off* aceptado y compensado en producción por el mecanismo de *forced-retry* y las guardas estructurales. Es una limitación declarada con transparencia, no ocultada.

Conclusión 3 (latencia tier-Alexa — CUMPLIDO). Tras la optimización por caché de prefijo, reglas *lean* y *streaming* de TTS, la latencia se ubicó holgadamente dentro del presupuesto de 4–5 s: la **mediana global por turno (p50) $\approx 1,22$ s** y las **acciones con invocación de herramienta $\approx 2,2$ s** (sobre el *replay* de 1.071 mensajes reales, el p50 fue $\approx 2,5$ s). Las colas problemáticas (Spotify) se mantuvieron por debajo del umbral catastrófico de 8 s (reducidas de 12,6 s a 5,6 s).

Conclusión 4 (operación local, privada y honesta — CUMPLIDO). El sistema opera 100 % offline, sin transmitir audio ni datos a la nube, y garantiza honestidad estructural mediante verificadores tri-estado que distinguen "no medible" de "falló" y guardas que

•

impiden afirmar efectos físicos no verificados. Esto satisface tanto el requisito de privacidad como el principio de que el asistente nunca debe mentir sobre lo que hizo.

Conclusión académica (lecciones aprendidas). La lección metodológica central es el valor del principio **"medir, no celebrar"**: ningún resultado se dio por bueno sin un número contra un gate definido de antemano, y los fallos se reportaron en crudo. Este rigor permitió descubrir causas raíz contraintuitivas (el *padding* del wake-word, el *busy-wait* de ONNX, la serialización corrupta del dataset) que un enfoque de "parchear el síntoma" habría enmascarado. La arquitectura por capas con *blast radius* contenido demostró que los bugs reales viven en los **bordes** (recovery, teardown, persistencia), no en el camino feliz —un hallazgo que orienta dónde invertir esfuerzo de testing en proyectos futuros.

Referencias

Sección única de referencias en formato **APA 7**, en orden alfabético, que consolida todas las fuentes citadas a lo largo del informe. Las fuentes de mercado, privacidad y literatura técnica de edge LLM fueron verificadas contra su página original el 2026-06-03 (autores y cifras contrastados). Las entradas marcadas **[VERIFICAR]** existen y se citan en el cuerpo, pero su entrada bibliográfica exacta (autoría, año o URL) debe reconfirmarse contra la fuente primaria antes de la impresión final, en coherencia con el principio metodológico del proyecto de no presentar como verificado lo que no se contrastó en esta revisión.

Android Developers. (s. f.). *Gemini Nano*. Google. Recuperado el 3 de junio de 2026, de <https://developer.android.com/ai/gemini-nano>

Astute Analytica. (2026, 10 de febrero). *Voice assistant market to reach US\$ 59.9 billion by 2033 driven by mass consumer adoption, enterprise voice AI, and smart device proliferation*. GlobeNewswire. <https://www.globenewswire.com/news-release/2026/02/10/3235286/0/en/Voice-Assistant-Market-to-Reach-US-59-9-Billion-by-2033-Driven-by-Mass-Consumer-Adoption-Enterprise-Voice-AI-and-Smart-Device-Proliferation-Astute-Analytica.html>

Bonatti, R., Zhao, D., Bonacci, F., Dupont, D., Abdali, S., Li, Y., Lu, Y., Wagle, J., Koishida, K., Bucker, A., Jang, L., & Hui, Z. (2024). *Windows Agent Arena: Evaluating multi-modal OS agents at scale* [Preprint]. arXiv. <https://arxiv.org/abs/2409.08264>

Cai, G., Tian, R., Yang, L., Jia, Y., Li, L., & Wang, J. (2026). Efficient inference for edge large language models: A survey. *Tsinghua Science and Technology*, 31(3). <https://doi.org/10.26599/TST.2025.9010166>

Google DeepMind. (s. f.). *Gemma — Model card* [tarjeta de modelo]. Google. Recuperado el 3 de junio de 2026, de <https://ai.google.dev/gemma> **[VERIFICAR: año/versión de la card]**

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software*, 12(6), 42–50. <https://doi.org/10.1109/52.469759>

•

LiveKit. (s. f.). *livekit-wakeword: An open-source wake word library* [biblioteca de software / modelo de detección de palabra de activación]. GitHub. Recuperado el 3 de junio de 2026, de <https://github.com/livekit/livekit-wakeword> **[VERIFICAR: año de la versión]**

MediaPipe (Google). (s. f.). *Hand landmarks detection guide* [documentación de modelo]. Google. Recuperado el 3 de junio de 2026, de https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker **[VERIFICAR]**

OpenAI. (2022). *Robust speech recognition via large-scale weak supervision (Whisper)* [tarjeta de modelo / repositorio]. <https://github.com/openai/whisper> **[VERIFICAR]**

Osterwalder, A., Pigneur, Y., & Clark, T. (2010). *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. John Wiley & Sons.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). *Generative agents: Interactive simulacra of human behavior* [Preprint]. arXiv. <https://arxiv.org/abs/2304.03442>

Project Management Institute. (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (7.^a ed.). Project Management Institute.

Rhasspy. (s. f.). *Piper: A fast, local neural text to speech system* [sistema de síntesis de voz / modelos VITS-ONNX]. GitHub. Recuperado el 3 de junio de 2026, de <https://github.com/rhasspy/piper> **[VERIFICAR: año de la versión]**

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum: The rules of the game*. Scrum.org. <https://scrumguides.org/>

Secure Data Recovery Services. (2024, 5 de agosto). *Listening in: Privacy concerns of voice assistants*. <https://www.securedatarecovery.com/blog/smart-device-privacy-concerns>

Villacura, E. (2026a). *BACKLOG_MAESTRO.md* [documento interno del proyecto Baxy]. Repositorio del proyecto.

Villacura, E. (2026b). *Baxy — Estado técnico y límites de hardware* [documento interno]. Repositorio del proyecto.

•

Villacura, E. (2026c). *vram_real_medida.csv* [conjunto de datos de mediciones]. Repositorio del proyecto.

Villacura, E. (2026d). *AUDITORIA_licencias.md* [auditoría de licencias de 511 paquetes, documento interno del proyecto Baxy]. Repositorio del proyecto.

Web Accessibility Initiative (W3C). (2023). *Web Content Accessibility Guidelines (WCAG) 2.2* [recomendación]. World Wide Web Consortium. <https://www.w3.org/TR/WCAG22/> **[VERIFICAR]**

Zhang, C., Huang, H., Ni, C., Mu, J., Qin, S., He, S., Wang, L., Yang, F., Zhao, P., Du, C., Li, L., Kang, Y., Jiang, Z., Zheng, S., Wang, R., Qian, J., Ma, M., Lou, J.-G., Lin, Q., Rajmohan, S., & Zhang, D. (2025). *UFO2: The desktop AgentOS* [Preprint]. arXiv. <https://arxiv.org/abs/2504.14603>

Zheng, Y., Chen, Y., Qian, B., Shi, X., Shu, Y., & Chen, J. (2024). *A review on edge large language models: Design, execution, and applications* [Preprint]. arXiv. <https://arxiv.org/abs/2410.11845>

Legislación (República de Chile).

Biblioteca del Congreso Nacional de Chile. (1970). *Ley N.º 17.336, sobre Propiedad Intelectual*. Diario Oficial de la República de Chile. <https://www.bcn.cl/leychile>

Biblioteca del Congreso Nacional de Chile. (1999). *Ley N.º 19.628, sobre Protección de la Vida Privada*. Diario Oficial de la República de Chile. <https://www.bcn.cl/leychile>

Biblioteca del Congreso Nacional de Chile. (2022). *Ley N.º 21.459, que establece normas sobre delitos informáticos*. Diario Oficial de la República de Chile. <https://www.bcn.cl/leychile>

Biblioteca del Congreso Nacional de Chile. (2024a). *Ley N.º 21.663, Ley Marco de Ciberseguridad*. Diario Oficial de la República de Chile. <https://www.bcn.cl/leychile>

Biblioteca del Congreso Nacional de Chile. (2024b). *Ley N.º 21.719, que regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales*. Diario Oficial de la República de Chile. <https://www.bcn.cl/leychile>